

**Smart Attendance and Classroom Access Platform with
Face Verification and Geofenced Check-In
PID 12**

Software Architecture Document

Version 1.0

Mentor: Mr. Pasan Sarathchandra

Group Members

230702K - WICKRAMAARACHCHI W.A.A.U.

230714A - WIJAYARATHNE T.W.O.A.

230734J - WITHANA W.D.P.S.

230736R - WITHANAGE M.H.

Revision History

Date	Version	Description	Author
09/08/2026	1.0	Initial complete version of the Software Architecture Document	Group 27

Table of Contents

1. Introduction	7
1.1 Purpose	7
1.2 Scope	7
1.3 Definitions, Acronyms, and Abbreviations	8
1.4 Overview	9
2. Architectural Representation	10
2.1 Architectural Views	11
2.2 Overall System Architecture	12
3. Architectural Goals and Constraints	12
3.1 Security and Privacy	13
3.2 Performance and Scalability	13
3.3 Reliability and Availability	13
3.4 Maintainability and Modularity	14
3.5 Technology and Implementation Constraints	14
3.6 Deployment and Portability Constraints	15
3.7 Data and Integration Constraints	15
3.8 Cost and Resource Constraints	15
3.9 Architectural Priorities	16
4. Use-Case View	16
4.1 Use-Case Realizations	22
4.1.1 User Login	22
4.1.2 Complete Initial Attendance Check-In	23
4.1.3 Create and Manage Attendance Session	24
4.1.4 Conduct In-Lecture Dynamic QR Verification	25
4.1.5 Review and Resolve Verification Case Manually	26
4.1.6 Manage Student Reference Face	27
5. Logical View	29
5.1 Overview	29
5.2 Architecturally Significant Design Packages	30
5.2.1 Identity and Access Package	30
5.2.2 Academic Management Package	30
5.2.3 Attendance Session Package	31
5.2.4 Attendance Verification Package	31
5.2.5 Face Verification Package	31
5.2.6 Notification Package	32
5.2.7 Reporting and Analytics Package	32
5.2.8 Supporting Packages	33
5.3 Core Domain Model	34

5.3.1 Identity and User Domain	35
5.3.2 Academic Domain	35
5.3.3 Attendance Domain	36
5.4 Important Domain Relationships	36
6. Process View	37
6.1 Process Organisation and Communication	37
6.2 Student Attendance Check-In Activity	38
6.3 Attendance Session Management Activity	40
6.4 Lecturer Creates Attendance Session Sequence	41
6.5 User Login Sequence	43
6.6 Student Attendance Check-In Sequence	44
6.7 Face Verification Sequence	45
6.8 Dynamic QR Validation Sequence	46
6.9 Geofence Validation Sequence	48
6.10 Manual Attendance Override Sequence	49
6.11 Administrator Manages Academic Data Sequence	50
6.11.1 Administrator Replaces Student Reference Face	50
6.11.2 Administrator Configures Classroom and Geofence	51
6.12 Attendance Session State	52
6.13 Attendance Record State	53
7. Deployment View	54
8. Implementation View	55
8.1 Overview	55
8.2 Layers	56
9. Data View (optional)	58
9.1 Entity-Relationship Model	58
9.2 Data-Store Responsibilities	63
10. Size and Performance	64
10.1 Overview	64
10.2 Dimensioning Characteristics	64
10.3 Performance Targets	64
11. Quality	65
11.1 Security and Privacy	65
11.2 Reliability and Availability	67
11.3 Performance and Scalability	67
11.4 Maintainability and Extensibility	67
11.5 Portability	67
11.6 Auditability	67
12. References	68

Individual Contributions

Member	Contribution
WICKRAMAARACHCHI W.A.A.U. 230702K	Introduction Architectural Representation Architectural Goals and Constraints
WIJAYARATHNE T.W.O.A. 230714A	Use-Case View Logical View
WITHANA W.D.P.S. 230734J	Process View Deployment View Quality
WITHANAGE M.H. 230736R	Process View Implementation View Data View Size and Performance

Member	Contribution (Diagrams)
WICKRAMAARACHCHI W.A.A.U. 230702K	User Login Sequence Diagram Student Attendance Check-In Sequence Diagram Face Verification Sequence Diagram Dynamic QR Validation Sequence Diagram Geofence Validation Sequence Diagram Manual Attendance Override Sequence Diagram
WIJAYARATHNE T.W.O.A. 230714A	Student Use-Case Diagram Lecturer Use-Case Diagram Administrator Use-Case Diagram Student Attendance Check-In Activity Diagram Attendance Session Management Activity Diagram Lecturer Creates Attendance Session Sequence Diagram
WITHANA W.D.P.S. 230734J	Attendance Session State Diagram Attendance Record State Diagram Entity-Relationship Diagram PostgreSQL and Redis Responsibility Diagram Deployment Diagram

	Security Architecture Diagram
WITHANAGE M.H. 230736R	Overall System Architecture Diagram Logical Service Decomposition Diagram Backend Package Diagram Overall Component Diagram Core Domain Class Diagram Administrator Manages Academic Data Sequence Diagram

Software Architecture Document

1. Introduction

The Smart Attendance and Classroom Access Platform with Face Verification and Geofenced Check-In is a web and mobile-based system designed to support secure and efficient university attendance management. Students use a mobile application to complete attendance check-in, while lecturers and administrators use a web dashboard to manage attendance sessions, academic data, verification cases, reports, and system settings.

The platform uses mandatory face verification and geofence validation during initial check-in, with optional dynamic QR verification during active lectures. This Software Architecture Document describes the platform's major architectural decisions, components, interfaces, runtime interactions, data stores, deployment structure, and security controls.

1.1 Purpose

The purpose of this Software Architecture Document is to provide an architectural overview of the Smart Attendance and Classroom Access Platform and record the significant design decisions made for the system.

It explains how the student mobile application, lecturer and administrator web dashboard, core backend, face-verification service, background workers, databases, storage services, and external providers work together. It also defines the responsibilities and boundaries of the main modules, including identity and access management, academic management, attendance-session management, attendance verification, face verification, notifications, reporting, and audit logging.

This document will support:

- Development and integration of the system components.
- Testing of major workflows and service interactions.
- Deployment and maintenance of the platform.
- Evaluation of the selected architecture.
- Future extension or scaling of individual components.

The intended audience includes the project development team, testers, supervisors, evaluators, system administrators, and future maintainers. The SRS defines what the system must do, while this document describes how the system is structured to satisfy those requirements.

1.2 Scope

This document applies to the complete architecture of the Smart Attendance and Classroom Access Platform.

It covers the following major system elements:

Student Mobile Application

The student mobile application supports authentication, profile and timetable viewing, trial face verification, attendance check-in, location capture, liveness checks, face-image capture, optional QR scanning, attendance history, and notifications.

Lecturer and Administrator Web Application

The web application allows lecturers to create, activate, monitor, close, and finalize attendance sessions. Lecturers can also initiate optional dynamic QR verification sessions, monitor live attendance, review failed or pending cases, manually update attendance with a reason, and generate reports.

Administrators can manage users, courses, course offerings, enrolments, timetables, classrooms, geofences, attendance policies, reference-face information, reports, and audit records.

Backend Components

The backend is implemented using FastAPI and follows a service-oriented modular architecture. The core backend is organised into separate functional modules within a single deployable application, while the Face Verification Service maintains a clear API boundary and is implemented and deployed separately. This separation isolates image preprocessing, embedding generation, and similarity comparison, which may require greater CPU and memory resources than the core backend operations.

The document also covers:

- Authentication and RBAC (Role-Based Access Control).
- Attendance-session and record management.
- Face, geofence, enrolment, timing, and QR validation.
- Retry and pending-review handling.
- PostgreSQL permanent data storage.
- Redis caching, short-lived state, counters, and job queues.
- Temporary image storage.
- Background workers.
- Push-notification delivery.
- Reporting, analytics, and audit logging.
- Security boundaries and encrypted communication.

The selected technologies include React Native with Expo, Next.js with TypeScript, FastAPI, PostgreSQL, Redis, OpenCV, DeepFace with ArcFace, cosine similarity, and Google ML Kit.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SAD	Software Architecture Document.
SRS	Software Requirements Specification.
API	Application Programming Interface.
REST API	Representational State Transfer API - an HTTP-based interface used for communication between software components.
HTTPS	Hypertext Transfer Protocol Secure - HTTP communication protected using TLS encryption.

RBAC	Role-Based Access Control - an authorization approach in which permissions are assigned according to user roles.
QR	Quick Response
GPS	Global Positioning System - satellite-based positioning system used to obtain location information.
Geofence	A virtual geographic boundary used to determine whether a device is within an allowed location.
Face Embedding	A numerical representation of facial characteristics used for similarity comparison.
Reference Embedding	An approved facial embedding associated with a student's registered identity and used as a comparison reference.
Liveness Check	A check intended to determine whether the captured face represents a live person rather than a simple spoof such as a photograph.
Cosine Similarity	A mathematical measure used to determine the similarity between facial embeddings.
Redis	An in-memory data store used for temporary state, caching, rate limiting, coordination, and background-job queues.
Object Storage	Storage designed for files such as images rather than structured relational records.
Background Worker	A process that performs asynchronous or queued tasks independently of the immediate API request.
Core Backend	The main FastAPI application containing the platform's functional backend modules, excluding the separately deployed Face Verification Service.
API Boundary	A defined interface separating software components or services and controlling how they communicate.
Audit Log	A persistent record of significant administrative, security, and attendance-related actions.
UUID	Universally Unique Identifier - an identifier used to uniquely identify records or objects.
Rate Limiting	A mechanism that restricts the number or frequency of requests within a defined period.

1.4 Overview

The remainder of this document is organized as follows:

Section 2 - Architectural Representation: Introduces the selected architecture and the architectural views used to describe it.

Section 3 - Architectural Goals and Constraints: Describes the security, privacy, performance, reliability, maintainability, deployment, and technology constraints that influence the architecture.

Section 4 - Use-Case View: Presents the major functions available to students, lecturers, and administrators.

Section 5 - Logical View: Describes the service module, domain classes, responsibilities, and dependencies.

Section 6 - Process View: Describes runtime workflows using activity, sequence, and state diagrams.

Section 7 - Deployment View: Shows the physical and cloud nodes on which the applications and services are deployed.

Section 8 - Implementation View: Describes the implementation components, packages, layers, and interfaces.

Section 9 - Data View: Presents the database structure and the division of responsibilities between PostgreSQL and Redis.

Section 10 - Size and Performance: Describes the significant capacity and performance considerations.

Section 11 - Quality: Explains how the architecture supports security, privacy, reliability, maintainability, scalability, availability, portability, and auditability.

2. Architectural Representation

The architecture of the Smart Attendance and Classroom Access Platform is represented through multiple complementary views. Each view describes the system from a different perspective so that its functionality, internal structure, runtime behaviour, implementation, data management, deployment, and security can be understood clearly.

The system follows a service-oriented modular architecture. The core backend is implemented as one deployable FastAPI application containing clearly separated functional modules. These modules manage identity and access, academic data, attendance sessions, attendance verification, notifications, reporting and analytics, and audit logging.

The computationally intensive Face Verification Service has a separate service boundary and can be deployed and scaled independently.

The student mobile application and the lecturer and administrator web dashboard communicate with the backend through REST APIs. PostgreSQL stores permanent and transactional data, while Redis supports caching, short-lived state, rate-limiting counters, and background-job queues. Temporary object storage is used for face images that require short-term processing or manual review. Background workers perform asynchronous operations such as face verification, notification delivery, and report generation.

2.1 Architectural Views

Architectural Views	Representation
Use-Case View	Describes the major system functions available to students, lecturers, and administrators using the Student, Lecturer, and Administrator Use-Case Diagrams.
Logical View	Describes the decomposition of the backend into service modules, and domain classes using the Logical Service Decomposition Diagram, and Core Domain Class Diagram.
Process View	Describes the runtime workflows and interactions among users, interfaces, services, workers, and data stores using activity, sequence, and state diagrams.
Deployment View	Describes the physical and cloud environments on which the applications, services, workers, and data stores are deployed using the Deployment Diagram.
Implementation View	Describes the major software components, their interfaces and their organization using the Overall Component Diagram, and Backend Package Diagram.
Data View	Describes the persistent data model and the division of storage responsibilities using the Entity-Relationship Diagram and PostgreSQL and Redis Responsibility Diagram.
Quality and Security View	Describes authentication, authorization, encryption, request validation, audit logging, protected data storage, and security boundaries using the Security Architecture Diagram.

2.2 Overall System Architecture

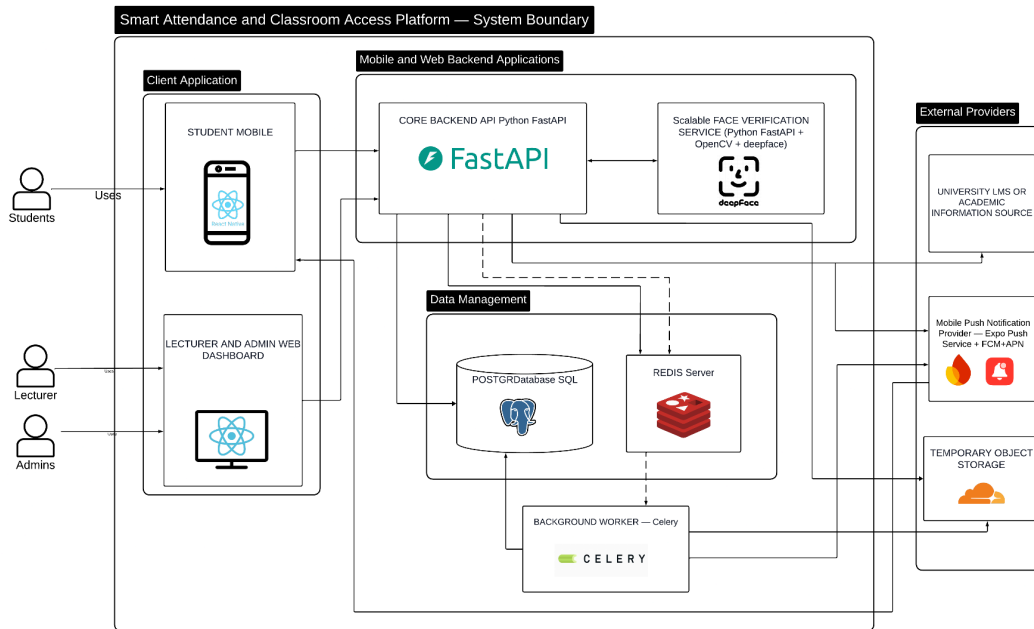


Figure 1. Overall System Architecture Diagram

Figure 1 presents the high-level architecture and system boundary of the Smart Attendance and Classroom Access Platform. The main client applications are the Student Mobile Application and the Lecturer and Administrator Web Dashboard. These applications communicate with the core backend API, which coordinates the platform's main attendance, academic, identity, notification, reporting, and audit functions.

The backend communicates with the separately deployable Face Verification Service to perform image preprocessing, face-embedding generation, and identity matching. PostgreSQL is used for permanent system data, Redis supports temporary and performance-sensitive operations, and object storage holds temporary face images. Background workers process queued tasks, while external services provide academic information and push-notification delivery.

The diagram also emphasizes that the core backend is deployed as one application containing separate service modules, while the Face Verification Service can be deployed and scaled independently due to its greater processing requirements.

3. Architectural Goals and Constraints

The architecture of the Smart Attendance and Classroom Access Platform is influenced by the need to provide secure, reliable, maintainable, and cost-effective attendance management while supporting face verification, geofence validation, dynamic QR verification, reporting, and real-time attendance monitoring.

3.1 Security and Privacy

The system processes sensitive information, including user credentials, attendance records, face embeddings, facial images, and location data. Therefore, security and privacy are major architectural goals.

The architecture shall support:

- Secure authentication using access and refresh tokens.
- RBAC for students, lecturers, and administrators.
- Encrypted communication using HTTPS/TLS.
- Secure password hashing.
- Input and request validation.
- Rate limiting and protection against repeated or malicious requests.
- Audit logging for administrative actions, verification results, and manual attendance updates.
- Restricted access to biometric and location-related information.
- Short retention periods for temporary face images.
- Collection of location information only during attendance-related actions.

Manual attendance updates must require an authorized lecturer, a reason for the change, and a corresponding audit-log entry. Students must also be informed about the use of face and location data, and consent must be obtained before these data are used for attendance verification.

3.2 Performance and Scalability

Attendance check-in is time-sensitive because many students may submit requests within a short period at the beginning of a lecture. The architecture must therefore minimize delays and prevent computationally expensive face-verification operations from overloading the main backend.

To support this goal:

- Reference face embeddings shall be generated in advance rather than recalculated during every check-in.
- Captured images shall be compressed in JPEG format and preprocessed before face comparison.
- Redis shall be used for caching, short-lived state, counters, and queued jobs.
- Background workers shall process face-verification requests at a controlled rate during peak periods.
- PostgreSQL queries shall use appropriate indexes and relational constraints.
- Lightweight validations, such as enrolment, session timing, geofence, and QR checks, shall be processed separately from computationally intensive face verification.

The Face Verification Service shall maintain a clear service interface and may be scaled independently if verification demand increases.

3.3 Reliability and Availability

The system must remain dependable during active attendance sessions because attendance actions are available only for limited periods.

The architecture shall:

- Prevent duplicate or conflicting attendance records.
- Use database transactions for critical updates.
- support retries for temporary network, image-quality, GPS, and backend failures.
- Place uncertain or repeatedly failed cases into a controlled manual-review process.
- Require a reason for every manual attendance update.
- Preserve audit information for changes to finalized or pending records.
- Support periodic database backups and recovery procedures.

PostgreSQL is used as the authoritative store for permanent data, while Redis is limited to temporary and recoverable data. Therefore, the loss of Redis data must not result in the loss of finalized attendance records.

3.4 Maintainability and Modularity

The system contains several distinct functional areas, including identity management, academic management, attendance sessions, attendance verification, face verification, notifications, reporting, and auditing.

A service-oriented modular architecture is selected for the core backend. This approach provides clear module boundaries without introducing the operational complexity of deploying every function as an independent microservice.

Each module shall:

- Have a clearly defined responsibility.
- Expose controlled interfaces to other modules.
- Avoid direct access to another module's internal implementation.
- Be independently testable.
- Maintain separate service, schema, and data-access logic where appropriate.

The Face Verification Service is separated from the core backend deployment because its resource requirements differ from ordinary API operations. This structure supports later replacement or independent scaling without redesigning the complete system.

3.5 Technology and Implementation Constraints

The implementation is constrained by the selected technology stack:

- React Native with Expo for the student mobile application.
- Next.js with TypeScript for the lecturer and administrator web application.
- Python FastAPI for the backend APIs.
- PostgreSQL for permanent relational data.
- Redis for caching, queues, counters, and short-lived state.
- OpenCV and DeepFace with a pretrained ArcFace model for face processing.
- Google ML Kit for mobile-side face detection and basic liveness support.
- REST APIs and JSON for client-server communication.

The selection of FastAPI allows the backend to integrate directly with Python-based computer-vision libraries without introducing a separate language bridge. PostgreSQL is required because the platform contains highly structured data and depends on relational integrity and transaction handling.

3.6 Deployment and Portability Constraints

The system is distributed across mobile devices, web hosting, backend compute, managed data stores, object storage, and external notification services.

The initial deployment is expected to use:

- Vercel for the Next.js web application.
- Expo Application Services for mobile builds and distribution.
- AWS EC2 for the FastAPI backend, Redis, and workers.
- Supabase-hosted PostgreSQL.
- Cloudflare R2 for temporary object storage.

Because free-tier limits, pricing, and platform availability may change, backend components shall be containerized and configured through environment variables. The architecture shall avoid unnecessary provider-specific dependencies so that components can be moved to alternative hosting platforms when required.

3.7 Data and Integration Constraints

The platform depends on student, lecturer, course, enrolment, timetable, and reference-photo information originating from existing university records. Full integration with the university ERP or LMS is not included in the current integration scope. Therefore the platform shall obtain the required information through controlled data imports from the university and LMS data sources.

The architecture must ensure that:

- External identifiers are mapped consistently to internal user and academic records.
- Imported reference photos are processed into face embeddings before attendance verification.
- One approved active reference embedding is maintained for each student.
- Permanent and transactional data are stored in PostgreSQL.
- Temporary QR state, cached sessions, counters, and queued jobs are stored in Redis.
- Temporary images are stored outside the relational database.

3.8 Cost and Resource Constraints

The architecture is designed to provide the required functionality while maintaining a practical and sustainable infrastructure cost. The platform primarily relies on open-source technologies, pretrained biometric models, standard computing infrastructure, and managed cloud services with cost-effective deployment options.

The initial deployment avoids specialised biometric hardware, custom model training, and unnecessary distributed infrastructure. These choices reduce infrastructure and operational costs while providing the

capabilities required for the current system scope.

The architecture also maintains clear component boundaries so that additional computing resources or infrastructure can be introduced as usage increases. This allows the platform to evolve toward higher-capacity or more specialised infrastructure without requiring fundamental changes to the core architecture.

3.9 Architectural Priorities

When architectural goals conflict, the following priorities shall guide design decisions:

1. Protection of student identity, biometric data, location data, and attendance records.
2. Correctness and auditability of attendance decisions.
3. Availability and acceptable response time during active lectures.
4. Maintainability and clear separation of responsibilities.
5. Portability and future scalability.
6. Reduction of infrastructure and operational complexity during the initial stage.

4. Use-Case View

The Use-Case View presents the main interactions between the Smart Attendance Platform and its three primary actors: Student, Lecturer, and Administrator. It focuses on functions that are central to the system or involve several architectural components, such as authentication, attendance verification, session management, manual review, academic-data management, reporting, and audit logging.

Separate use-case diagrams are provided for each actor to keep the responsibilities and system functions clear. The selected use cases are later realised through the mobile application, web dashboard, backend service modules, Face Verification Service, PostgreSQL, Redis, object storage, background workers, and external providers.

Student Use Cases

The student interacts with the system primarily through the mobile application.

Use case	Brief description
Log In	Authenticates the student and provides access to the mobile application.
View and Update Profile	Allows the student to view personal information and update permitted profile details.
View Upcoming Sessions	Displays upcoming lectures and available attendance sessions.
Complete Trial Face Verification	Allows the student to test whether the registered reference face can be verified successfully.
Complete Initial Attendance Check-In	Records the student's initial attendance using face verification, liveness evidence, geofence validation, enrolment validation, and session timing.

Complete In-Lecture QR Verification	Allows the student to respond to an optional dynamic QR-verification session opened by the lecturer.
View Attendance History	Displays previous attendance records and their final statuses.
View Notifications and Alerts	Displays upcoming-session reminders and attendance-related notifications.

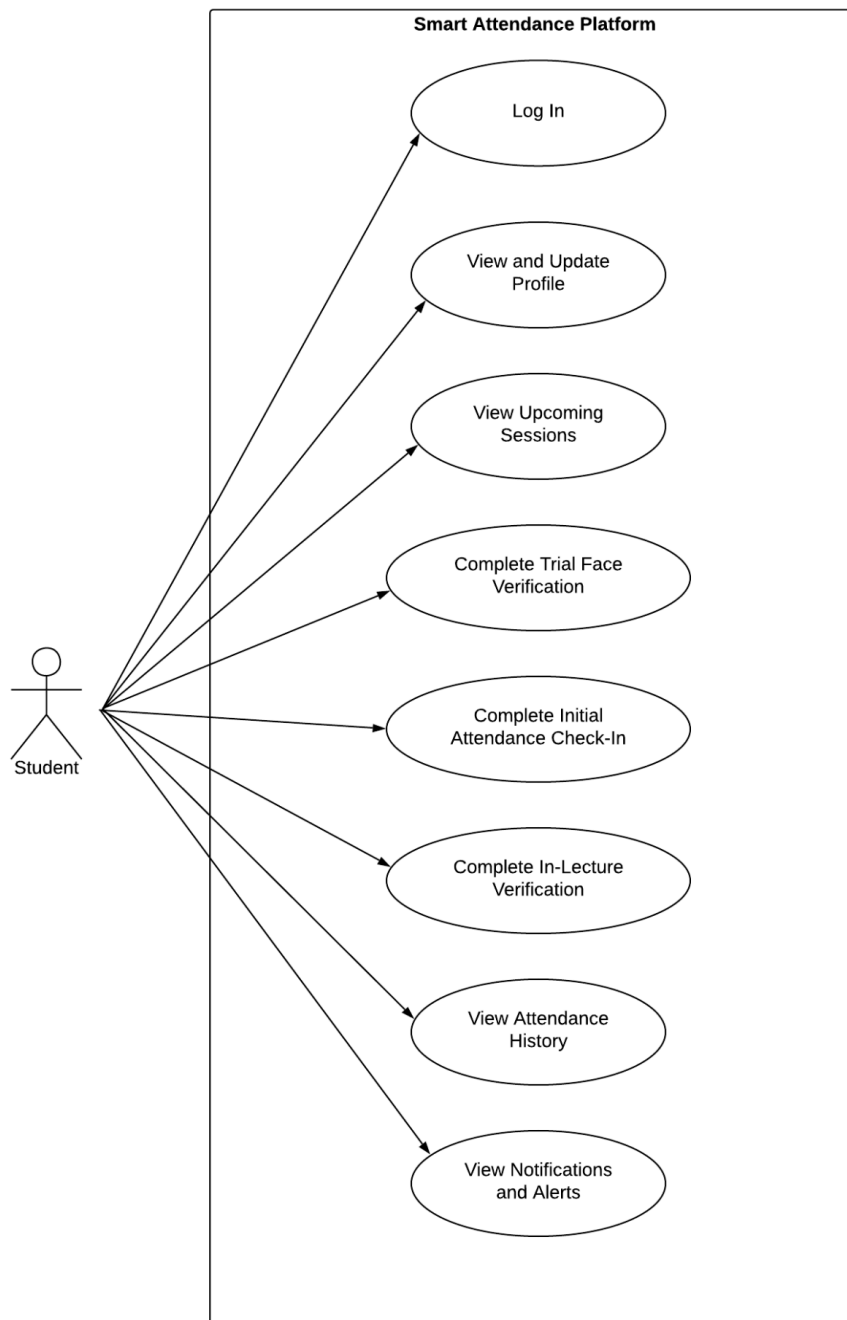


Figure 2. Student Use-Case Diagram

Figure 2 shows the main functions available to a student through the mobile application. The architecturally significant student functions are initial attendance check-in, trial face verification, and optional in-lecture QR verification because they involve mobile-device features, backend validation, verification services, and persistent data storage.

Lecturer Use Cases

The lecturer interacts with the system through the web dashboard.

Use case	Brief Description
Log In	Authenticates the lecturer and provides access to lecturer functions.
View and Update Profile	Allows the lecturer to view and update permitted profile information.
Create and Manage Attendance Sessions	Allows the lecturer to create, activate, monitor, close, and finalize attendance sessions for assigned lectures.
Monitor Live Attendance	Displays the current attendance and verification progress of students during an active session.
Initiate Dynamic QR Verification	Allows the lecturer to open one or more optional, short-lived QR-verification sessions during an active lecture.
Review and Resolve Verification Cases Manually	Allows the lecturer to review failed or uncertain verification cases and update attendance with a recorded reason.
View Reports and Analytics	Provides course-level attendance reports, summaries, late-arrival information, and attendance trends.

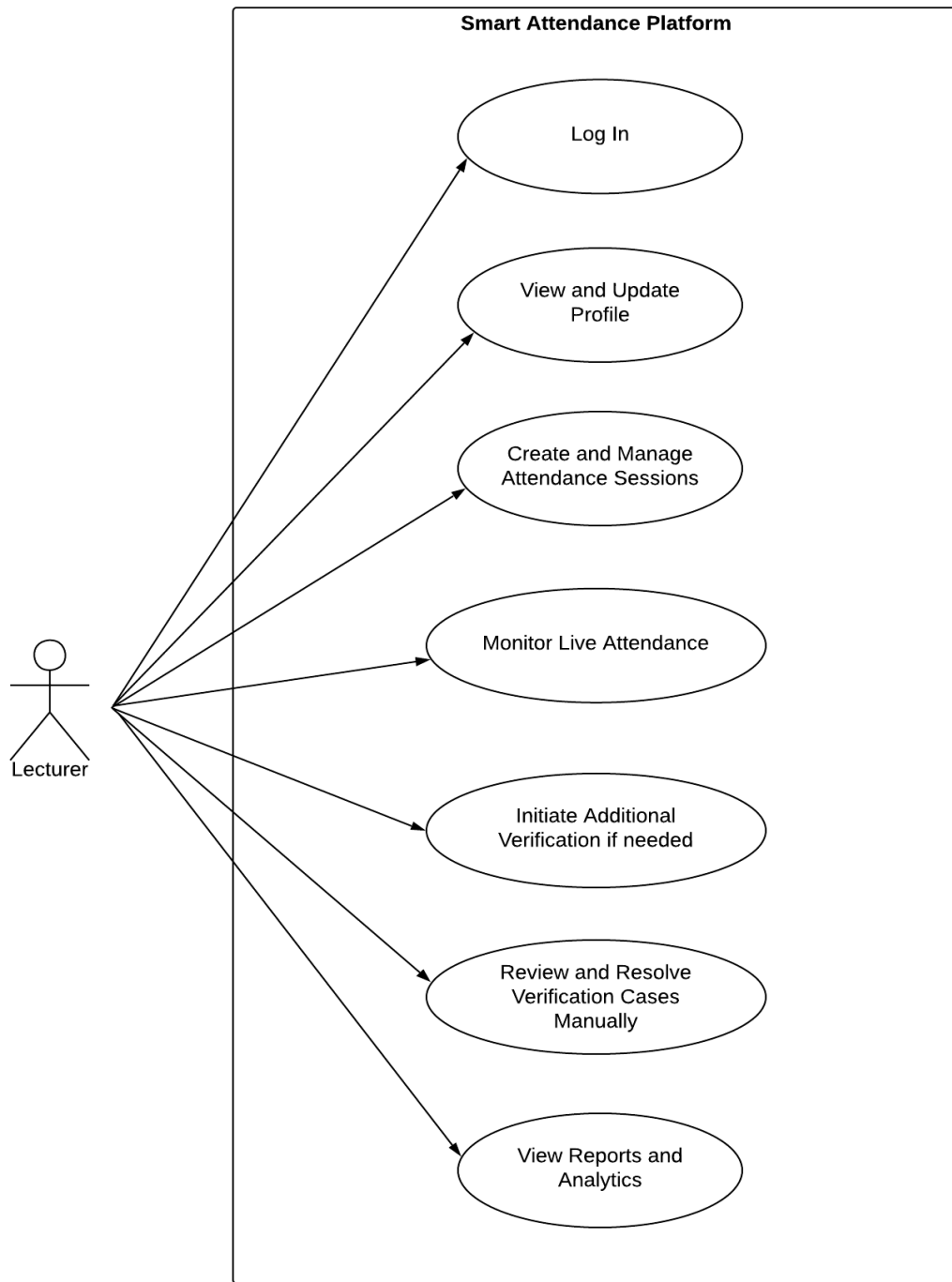


Figure 3. Lecturer Use-Case Diagram

Figure 3 presents the functions available to a lecturer. Attendance-session management has broad architectural coverage because it interacts with academic data, session configuration, verification evidence, live attendance data, notification handling, reporting, and audit logging.

Administrator Use Cases

The administrator uses the web dashboard to manage institution-level information and system configurations.

Use Case	Brief Description
Log In	Authenticates the administrator and provides access to administrative functions.
Create and Manage User Profiles	Manages student, lecturer, and administrator accounts and their statuses.
Synchronise and Manage Academic Data	Manages courses, course offerings, academic periods, lecturer assignments, timetables, and enrolments.
Manage Classrooms and Geofences	Configures classroom information, geographic coordinates, permitted radius, and related settings.
Configure System and Attendance Policies	Configures check-in windows, late thresholds, verification attempts, and other attendance rules.
Manage Student Reference Faces	Manages the approved reference-face information and active face embeddings used for student verification.
View Reports and Analytics	Provides institution-level attendance reports and analytical summaries.
Review Audit Logs	Allows the administrator to review important security, administrative, verification, and attendance-related actions.

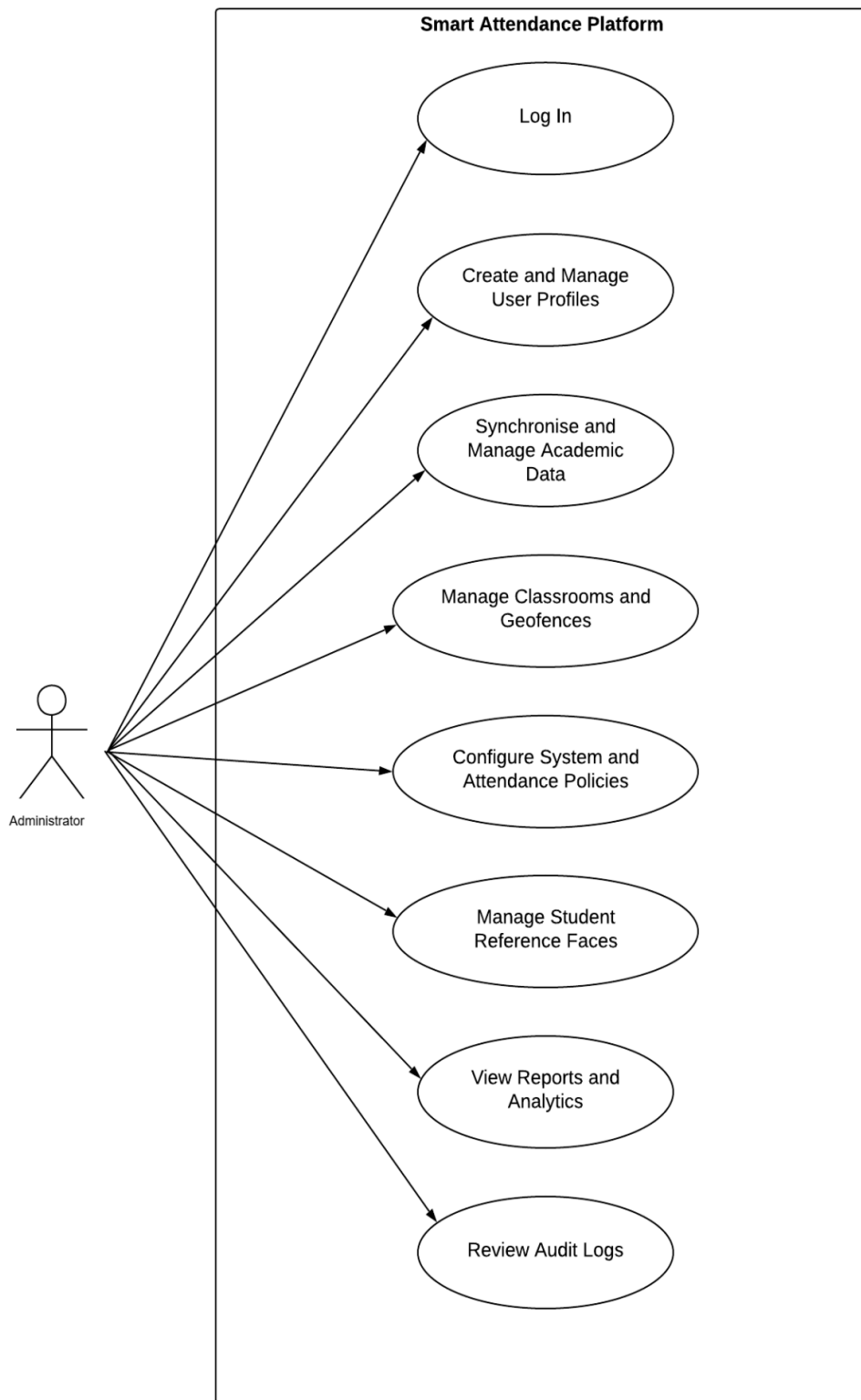


Figure 4. Administrator Use-Case Diagram

Figure 4 shows the administrative functions required to configure and operate the platform. Academic-data, geofence, reference-face, policy, and audit-log management are significant because they provide the trusted data and configuration used by the attendance-verification process. The three use-case diagrams used in this section are included in the project diagram collection.

4.1 Use-Case Realizations

The following realizations describe selected use cases that exercise several architectural elements or represent security-sensitive and operationally important workflows. Supporting activity, sequence, and state diagrams for these realizations are presented later in the Process View.

4.1.1 User Login

Use case name	User Login
Actor	Student, Lecturer, Administrator
Description	Authenticates a user and provides access to the interface and functions permitted for the user's role.
preconditions	The user has an existing account. The account has not been disabled or locked. The mobile application or web dashboard can communicate with the backend.
Main flow	1. The user opens the mobile application or web dashboard. 2. The user enters the username and password. 3. The client submits the credentials to the Identity and Access Service through the backend API. 4. The service retrieves the user account and verifies the password, account status, and assigned role. 5. When a first-login password change is required, the user is prompted to enter a new password. 6. The service creates the access and refresh tokens. 7. The client stores the required authentication information securely. 8. The user is redirected to the student, lecturer, or administrator interface according to the assigned role.
Successful end/post condition	The user has an authenticated session and can access functions permitted for the assigned role.
Fail end/post condition	Access is denied, no authentication tokens are issued, and an appropriate error is displayed. Failed-login information may be recorded for security control purposes.
Extensions	Invalid credentials cause the attempt to fail. A disabled or locked account is denied access. Repeated failed attempts may trigger rate limiting or temporary lockout. A user who has forgotten the password may use the password-reset process.

Contributing design elements:

The client interface collects the credentials, while the Identity and Access Service performs password, account-status, and role validation. PostgreSQL stores the account and role information, and Redis may maintain temporary rate-limiting or failed-login counters.

4.1.2 Complete Initial Attendance Check-In

Use case name	Complete Initial Attendance Check-In
Actor	Student
Description	Verifies the student's identity and location and records initial attendance for an active lecture session.
preconditions	The student is logged in, enrolled in the relevant course offering, and has an active reference-face embedding. The attendance session is active and its initial check-in window is open. Camera and location permissions are available. The student has not already completed the initial check-in.
Main flow	1. The student selects the active attendance session and starts check-in. 2. The mobile application requests the session details from the backend. 3. The Attendance Verification Service confirms that the session is active, the student is enrolled, the check-in window is open, and no successful initial check-in already exists. 4. The mobile application obtains the current device location and its reported accuracy. 5. The backend compares the submitted location with the session geofence. 6. The mobile application presents a randomized liveness challenge. 7. The student completes the challenge and captures a face image. 8. The application submits the location, liveness result, and face image to the backend. 9. The Face Verification Service checks image quality, detects and processes the face, generates an embedding, and compares it with the registered reference embedding. 10. The Attendance Verification Service evaluates the face, geofence, liveness, enrolment, and session-timing evidence. 11. A successful result is classified as Present or Late according to the configured threshold. 12. The attendance record and verification logs are stored, and the result is displayed to the student.
Successful end/post condition	A valid attendance record is stored as On-time or Late, together with the relevant verification evidence and check-in time.

Fail end/post condition	A successful attendance status is not assigned. The failed attempt is recorded, and the student is allowed to retry or the case is placed in Pending Review according to the configured policy.
Extensions	Poor image quality causes the application to request another capture. A location near the geofence boundary may require review. If face processing is queued, the application displays a pending status. Temporary network or backend failures may allow a retry within the permitted period. When all attempts are exhausted, the student may request lecturer review.

Contributing design elements:

This realization involves the Student Mobile Application, Attendance Session Service, Attendance Verification Service, Face Verification Service, Academic Management Service, PostgreSQL, Redis, temporary object storage, and background workers. It is one of the most architecturally significant workflows because it combines mobile permissions, location processing, biometric verification, timing rules, and transactional attendance recording.

4.1.3 Create and Manage Attendance Session

Use case name	Create and Manage Attendance Session
Actor	Lecturer
Description	Allows an assigned lecturer to prepare, activate, monitor, close, and finalize an attendance session for a scheduled lecture.
preconditions	The lecturer is logged in and assigned to the relevant course offering or timetable entry. The related course, timetable, classroom, geofence, and attendance-policy information exists.
Main flow	1. The lecturer selects an assigned timetable entry. 2. The web dashboard requests the available session-creation data and default settings. 3. The lecturer reviews the session timing, check-in window, late threshold, classroom, geofence, and other permitted settings. 4. The lecturer submits the configuration. 5. The Attendance Session Service validates lecturer authorization, timetable information, session times, and duplicate-session conditions. 6. The system stores the session in the Scheduled state. 7. At the permitted time, the lecturer activates the session. 8. The system changes the session to Active, opens the initial check-in window, and creates the required student attendance entries or provisional records. 9. The lecturer monitors live attendance and may initiate optional dynamic QR verification. 10. The lecturer closes the session, or the system closes it according to the configured timing rules. 11. The system stops further submissions and evaluates the

	collected attendance evidence. 12. The lecturer reviews any unresolved cases and confirms finalization. 13. Final attendance statuses are calculated, stored, and made available for reporting.
Successful end/post condition	The attendance session is finalized, its student attendance records are stored, and the session results are available for students, reports, and analytics.
Fail end/post condition	The session is not created, activated, or finalized. The existing session state remains unchanged, and validation errors are displayed.
Extensions	The lecturer may correct invalid configuration and resubmit it. A scheduled session may be cancelled before activation. A session that is not opened may expire. The system may close an active session automatically at the configured end time. Sessions containing unresolved cases may enter a Manual Review Pending state before finalization.

Contributing design elements:

The Web Dashboard communicates with the Attendance Session Service, Academic Management Service, Attendance Verification Service, Notification Service, Reporting and Analytics Service, Audit Service, and PostgreSQL. Session status and temporary live-session information may also be cached in Redis.

4.1.4 Conduct In-Lecture Dynamic QR Verification

Use case name	Conduct In-Lecture Dynamic QR Verification
Actor	Lecturer, Student
Description	Allows the lecturer to conduct an optional, time-limited QR-verification session during an active attendance session and allows students to submit proof of classroom presence.
preconditions	The attendance session is Active. The lecturer is authorized to manage the session. The student is logged in and enrolled in the relevant course offering.
Main flow	1. The lecturer starts a dynamic QR-verification session from the web dashboard. 2. The Attendance Session Service creates a short-lived QR session associated with the active attendance session. 3. The backend generates a digitally signed QR token containing the required session and expiry information. 4. The web dashboard displays the QR code and refreshes it periodically while the QR session remains active. 5. The student scans the QR code using the mobile application. 6. The application submits the token and relevant session information to the backend.

	7. The backend verifies the token signature, linked session, server-time validity, expiry, and previous usage. 8. Any additional location requirement configured for the verification is evaluated. 9. The system stores the QR-verification result and associates it with the student's attendance evidence. 10. The result is displayed to the student and is available to the lecturer's live view.
Successful end/post condition	A valid QR-verification result is stored for the student and associated with the correct attendance and QR sessions.
Fail end/post condition	The QR result is rejected and no successful evidence is recorded for the submitted token. The reason for failure is returned to the application.
Extensions	An expired token may be replaced by scanning the currently displayed token while the QR session remains open. An invalid, modified, duplicated, or replayed token is rejected. The lecturer may open multiple separate QR-verification sessions during the same active attendance session.

Contributing design elements:

This realization involves the Lecturer Web Dashboard, Student Mobile Application, Attendance Session Service, Attendance Verification Service, Redis, and PostgreSQL. Redis is suitable for short-lived token state and replay-prevention information, while permanent verification results are stored in PostgreSQL.

4.1.5 *Review and Resolve Verification Case Manually*

Use case name	Review and Resolve Verification Case Manually
Actor	Lecturer
Description	Allows an authorized lecturer to review failed or uncertain attendance evidence and assign an appropriate attendance status with a documented reason.
preconditions	The lecturer is logged in and assigned to the relevant course or attendance session. A failed, uncertain, or Pending Review attendance case exists.
Main flow	1. The lecturer opens the list of failed and pending verification cases. 2. The web dashboard requests the selected attendance record and its available verification evidence. 3. The backend retrieves the attendance record, face result, geofence result, timing information, attempt history, and other relevant evidence. 4. The lecturer reviews the information and selects an appropriate attendance status. 5. The lecturer enters a mandatory reason for the decision.

	6. The request is submitted to the Attendance Verification Service. 7. The backend verifies the lecturer's authorization and validates the requested status change. 8. The attendance record is updated within a database transaction. 9. The Audit Service records the lecturer, student, session, previous status, new status, reason, and timestamp. 10. The updated result is displayed and included in the final session evaluation.
Successful end/post condition	The attendance record is updated and a complete audit entry is stored for the manual decision.
Fail end/post condition	The attendance record remains unchanged. The rejected request and its reason are returned to the web dashboard and may be logged.
Extensions	The lecturer may leave the case in Pending Review when insufficient evidence exists. Unauthorized requests are rejected. A missing reason, invalid status transition, or conflicting record update prevents the change. An administrator may later review the corresponding audit entry.

Contributing design elements:

The realization involves the Lecturer Web Dashboard, Identity and Access Service, Attendance Verification Service, Audit Service, and PostgreSQL. Authorization checks and mandatory audit logging ensure that manual changes remain controlled and traceable.

4.1.6 *Manage Student Reference Face*

Use case name	Manage Student Reference Face
Actor	Administrator
Description	Allows an authorized administrator to replace an outdated or invalid student reference face and generate a new active face embedding.
preconditions	The administrator is logged in and has permission to manage biometric reference information. The student exists in the system, and an approved replacement image is available from the authorized university source or process.
Main flow	1. The administrator searches for the student using the registration or index number. 2. The system retrieves the student's reference information and current biometric-profile status. 3. The administrator selects the approved replacement image and enters the required reason or source information. 4. The image is submitted to the backend. 5. The backend validates the administrator's authorization and confirms the student identity.

	6. The Face Verification Service checks image quality and confirms that exactly one usable face is present. 7. The service preprocesses the face and generates a new ArcFace embedding. 8. The new embedding and its model metadata are returned to the backend. 9. The previous reference is deactivated, and the new reference embedding is stored as the active reference. 10. The Audit Service records the replacement action. 11. The updated biometric-profile status is displayed to the administrator.
Successful end/post condition	The student has one valid active reference embedding, and the replacement action is recorded for auditing.
Fail end/post condition	The existing active reference remains unchanged. The invalid replacement is not activated, and an error is displayed.
Extensions	An image containing no face, multiple faces, poor quality, or an unsupported format is rejected. If embedding generation fails, the administrator may upload another approved image. The administrator may deactivate an invalid reference until an approved replacement becomes available.

Contributing design elements:

This realization involves the Administrator Web Dashboard, Identity and Access Service, Academic Management Service, biometric-profile management logic, Face Verification Service, PostgreSQL, and Audit Service. It is architecturally significant because it modifies sensitive biometric reference data used by all later face-verification requests.

5. Logical View

The Logical View describes how the system is divided into functional modules, packages, and domain classes. It focuses on the responsibilities of each part of the system and the important relationships between them rather than their physical deployment.

The core backend is organised as a service-oriented modular monolith. Each service has a defined responsibility and communicates with other services through controlled interfaces. The Face Verification Service belongs to the overall logical design but maintains a separate service boundary because it may be deployed independently.

5.1 Overview

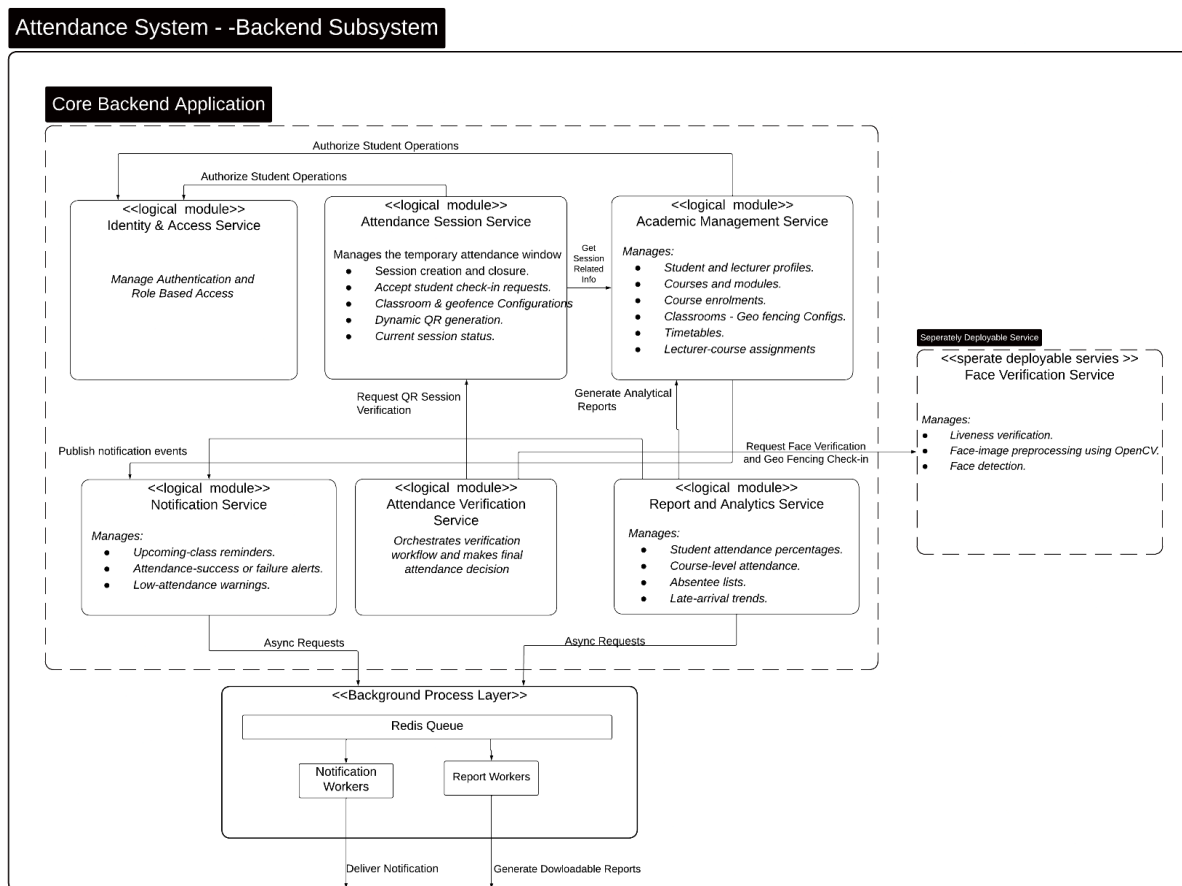


Figure 5. Logical Service Decomposition Diagram

Figure 5 shows the decomposition of the platform into eight main service areas. Seven services form the core backend application, while the Face Verification Service is separated by a clear API boundary. Background workers support asynchronous operations such as notification delivery, report generation, and queued verification processing.

The main service relationships are as follows:

- The Identity and Access Service authenticates users and authorises operations.
- The Academic Management Service provides student, lecturer, course, enrolment, timetable,

- classroom, and geofence information.
- The Attendance Session Service manages attendance-session and QR-verification lifecycles.
- The Attendance Verification Service coordinates the checks required to determine attendance results.
- The Face Verification Service performs face-image processing and identity comparison.
- The Notification Service distributes attendance-related alerts.
- The Reporting and Analytics Service produces summaries and analytical information.
- The Audit Service records sensitive and important system actions.

The logical decomposition diagram is one of the three diagrams assigned to the Logical View in the project diagram collection.

Main service dependencies

The Attendance Verification Service acts as the main coordinator during student attendance processing. It obtains session information from the Attendance Session Service, confirms enrolment and academic information through the Academic Management Service, requests identity comparison from the Face Verification Service, and stores the final attendance decision.

The Attendance Session Service uses academic information when creating a session from an assigned timetable entry. It also manages active-session details, check-in periods, late thresholds, session geofences, QR-verification sessions, and dynamic QR-token information.

The Notification and Reporting services use information produced by the other modules but do not control the attendance decision. The Audit Service receives records of security-sensitive, administrative, and manual attendance actions.

5.2 Architecturally Significant Design Packages

5.2.1 Identity and Access Package

The Identity and Access package manages user authentication and access control.

Its main responsibilities are:

- Login and logout.
- Access-token and refresh-token handling.
- Password activation, change, and reset.
- User-account status and failed-login handling.
- Role and permission validation.
- Student, lecturer, and administrator profile association.
- Consent-status management.

This package is used by all protected backend operations to identify the current user and determine whether the requested action is permitted.

5.2.2 Academic Management Package

The Academic Management package manages the academic information required by the attendance process.

Its main responsibilities are:

- Student and lecturer profiles.
- Academic periods.
- Courses and course offerings.
- Student enrolments.
- Lecturer assignments.
- Timetables.
- Classrooms.
- Classroom coordinates and default geofence configurations.

The Attendance Session package uses this information when preparing sessions, while the Attendance Verification package uses it to confirm student eligibility and enrolment.

5.2.3 *Attendance Session Package*

The Attendance Session package manages the lifecycle and configuration of attendance sessions.

Its main responsibilities are:

- Creating scheduled attendance sessions.
- Activating, closing, cancelling, and finalising sessions.
- Managing check-in windows and late thresholds.
- Associating sessions with classrooms and geofences.
- Maintaining the expected student list.
- Managing optional dynamic QR-verification sessions.
- Generating and validating short-lived QR-token state.
- Maintaining session-status history.

This package determines whether an attendance action is currently permitted but does not make the complete final attendance decision.

5.2.4 *Attendance Verification Package*

The Attendance Verification package coordinates the verification workflow and determines attendance results.

Its main responsibilities are:

- Confirming that the attendance session is active.
- Confirming that the student is enrolled.
- Preventing duplicate attendance submissions.
- Coordinating face, geofence, QR, liveness, and timing checks.
- Recording verification attempts and their individual results.
- Assigning Present or Late after successful initial check-in.
- Identifying cases that require manual review.
- Finalising attendance records according to the attendance policy.
- Supporting controlled manual attendance updates.

This package contains the central application logic for attendance decisions.

5.2.5 *Face Verification Package*

The Face Verification package performs biometric identity verification through a separate service interface.

Its main responsibilities are:

- Validating face-image quality.
- Detecting a usable face.
- Rejecting images containing no face or multiple faces.
- Cropping, aligning, resizing, and normalising the face image.
- Generating an ArcFace embedding.
- Retrieving the student's active reference embedding.
- Comparing embeddings using cosine similarity.
- Returning the match result and similarity score.
- Managing reference-embedding generation and replacement.

The package exposes a clear interface so that its implementation or deployment can be modified without changing the attendance workflow.

5.2.6 Notification Package

The Notification package manages system-generated notifications.

Its main responsibilities are:

- Upcoming-session reminders.
- Attendance-session opening alerts.
- Dynamic QR-session alerts.
- Verification-result notifications.
- Attendance-related warnings.
- Notification preferences.
- Mobile-device tokens.
- Push- and email-notification status tracking.

Notification delivery may be executed by background workers so that it does not delay the main attendance operations.

5.2.7 Reporting and Analytics Package

The Reporting and Analytics package provides attendance summaries and analysis.

Its main responsibilities are:

- Student attendance history.
- Course and session attendance summaries.
- Attendance percentages.
- Present, late, absent, and left-early counts.
- Low-attendance identification.
- Late-arrival trends.
- Lecturer and administrator dashboards.
- Downloadable reports.

This package reads final attendance and academic information without modifying the underlying attendance decisions.

5.2.8 *Supporting Packages*

The business packages use the following supporting packages:

Supporting package	Responsibility
Database	Database connection management, migrations, shared database utilities, transactions, and common persistence functions.
Background Tasks	Notification tasks, report-generation tasks, and other asynchronous operations.
Integrations	Communication with university information sources, object storage, notification providers, and the external Face Verification Service.
Common Configuration	Environment configuration, logging, shared constants, common exceptions, and reusable utilities.

Business logic should remain inside the relevant service package rather than being placed in shared utility packages.

5.3 Core Domain Model

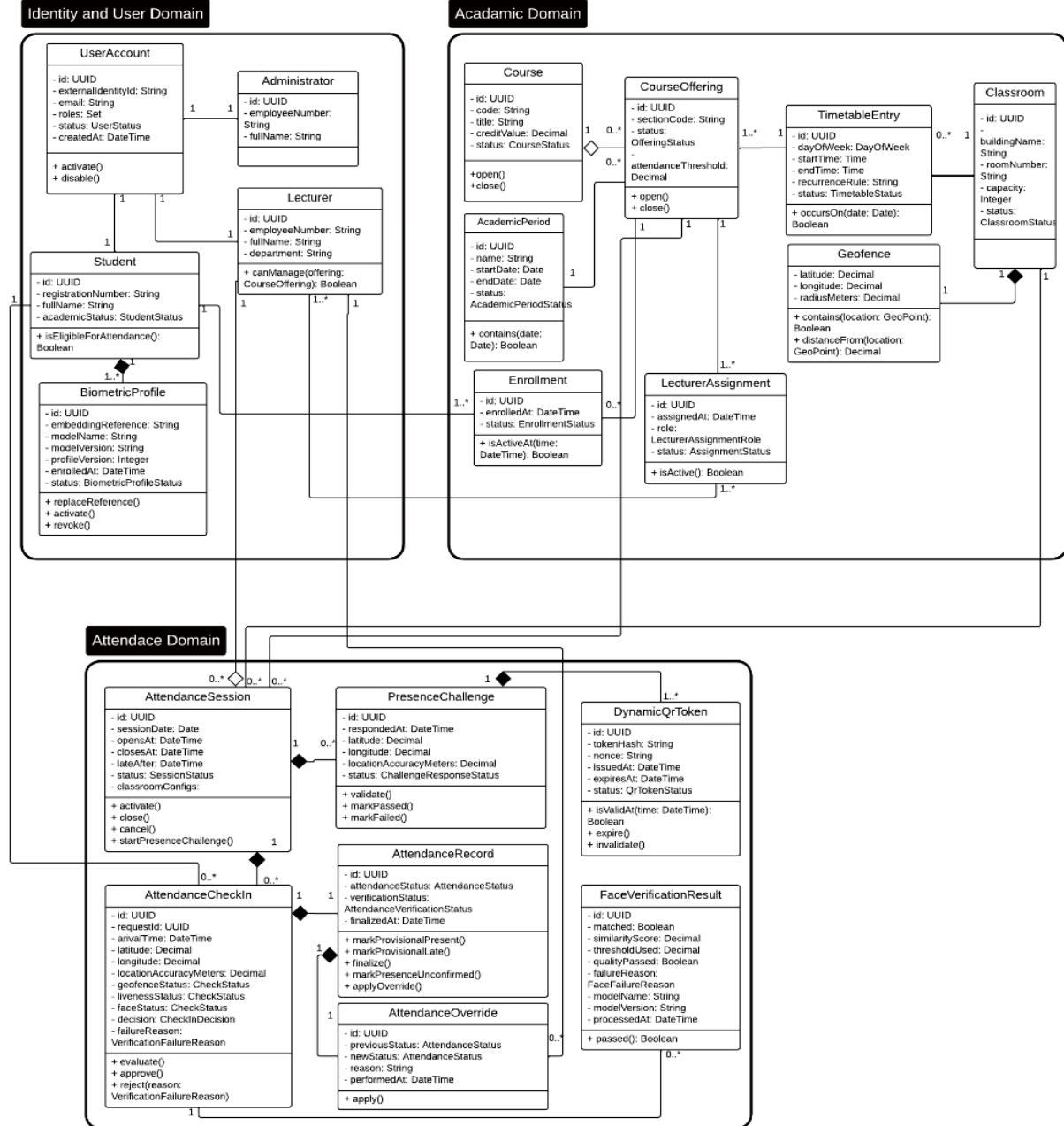


Figure 6. Core Domain Class Diagram

Figure 6 presents the architecturally significant domain classes and their relationships. The classes are grouped into the Identity and User Domain, Academic Domain, and Attendance Domain.

5.3.1 Identity and User Domain

The main classes in this domain are:

Class	Responsibility
UserAccount	Represents the common authentication account, account status, external identity, and assigned roles.
Student	Represents a student and the academic identity used for enrolment and attendance.
Lecturer	Represents a lecturer and their association with assigned course offerings.
Administrator	Represents an administrative user with system-management privileges.
BiometricProfile	Represents the student's reference-face profile, active embedding, model information, and biometric-profile status.

Student, Lecturer, and Administrator are role-specific profiles associated with a UserAccount. A student is associated with the biometric profile used during face verification.

5.3.2 Academic Domain

The main classes in this domain are:

Class	Responsibility
AcademicPeriod	Represents a semester or other defined academic period.
Course	Represents a course and its general academic information.
CourseOffering	Represents a course delivered during a specific academic period.
Enrollment	Associates a student with a course offering.
LecturerAssignment	Associates a lecturer with a course offering.
TimetableEntry	Defines the scheduled date, time, classroom, and recurrence of a lecture.
Classroom	Represents a physical lecture location.
Geofence	Defines the coordinates and permitted radius associated with a classroom or attendance session.

A course may have multiple course offerings across different academic periods. Students and lecturers are connected to a course offering through enrolments and lecturer assignments. Timetable entries identify when and where sessions may be conducted.

5.3.3 Attendance Domain

The main classes in this domain are:

Class	Responsibility
AttendanceSession	Represents a scheduled attendance event and its lifecycle, timing, and configuration.
AttendanceCheckIn	Represents the student's submitted attendance attempt and collected evidence.
AttendanceRecord	Represents the final or provisional attendance result for one student in one session.
PresenceChallenge	Represents an optional classroom-presence verification event conducted during the session.
DynamicQRToken	Represents a short-lived token generated for a QR-verification session.
FaceVerificationResult	Represents the outcome, score, threshold, and failure information of face verification.
AttendanceOverride	Represents an authorised manual change to an attendance record.

An AttendanceSession is created from an academic timetable context and is associated with multiple enrolled students. Each student may produce verification attempts and one attendance record for the session. Face, geofence, QR, and timing evidence contributes to the final AttendanceRecord. A manual change is represented by an AttendanceOverride and must retain the previous status, updated status, reason, responsible lecturer, and timestamp.

5.4 Important Domain Relationships

The most important logical relationships are:

- A UserAccount is associated with a Student, Lecturer, or Administrator profile.
- A Student is enrolled in one or more CourseOfferings through Enrollment.
- A Lecturer is assigned to one or more CourseOfferings through LecturerAssignment.
- A CourseOffering contains timetable entries and produces attendance sessions.
- A TimetableEntry is associated with a Classroom.
- A Classroom provides the default Geofence configuration.
- An AttendanceSession contains attendance evidence and records for multiple students.
- An AttendanceRecord belongs to one student and one attendance session.
- An AttendanceCheckIn contributes face, location, timing, and optional QR evidence.
- A DynamicQRToken belongs to a QR-verification event within an active attendance session.
- An AttendanceOverride updates an AttendanceRecord and produces a corresponding audit entry.

These relationships maintain the separation between user identity, academic organisation, attendance-session control, verification evidence, and the final attendance decision.

6. Process View

6.1 Process Organisation and Communication

The main runtime processes are organised into the following groups:

Process group	Main participants	Communication
User authentication	Mobile application or web dashboard, Authentication API, PostgreSQL	Synchronous API requests and database queries
Student attendance check-in	Student, mobile application, backend API, attendance services, verification services, database	Synchronous API requests with coordinated validation
Face verification	Mobile application, backend API, Face Service, database	Service request-response communication and database access
Geofence validation	Mobile application, backend API, Geofence Service, database	Location submission, configuration retrieval, and validation response
Dynamic QR validation	Lecturer dashboard, student mobile application, backend API, database	QR-session creation, token submission, and server-side validation
Attendance-session management	Lecturer, web dashboard, attendance backend, PostgreSQL	Synchronous commands, state updates, and live-status retrieval
Manual attendance update	Lecturer dashboard, backend API, Attendance Service, database, audit log	Authorised transactional update and audit recording
Administrative management	Administrator, web application, backend services, Face Verification Service, PostgreSQL	Authorised API calls and persistent data updates

6.2 Student Attendance Check-In Activity

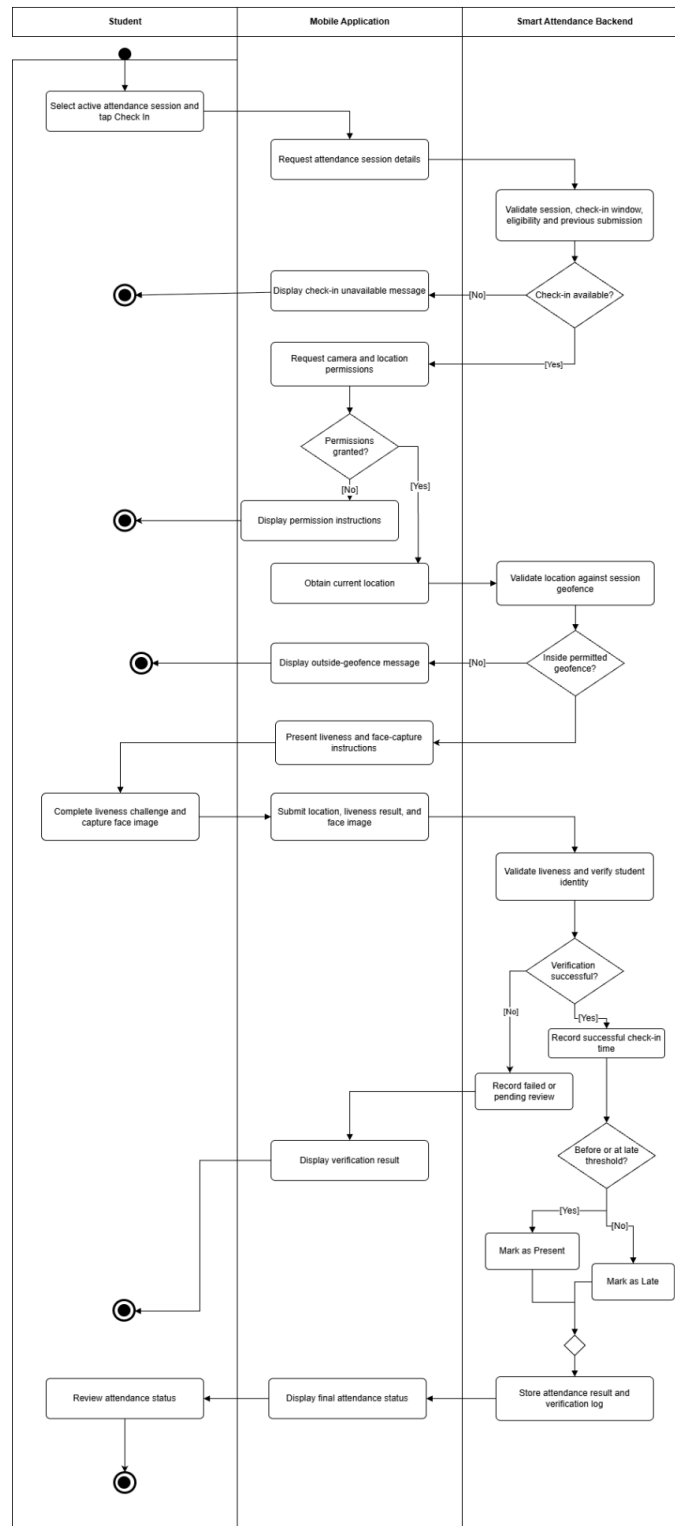


Figure 7. Student Attendance Check-In Activity Diagram

Figure 7 presents the complete initial attendance check-in workflow across the Student, Mobile Application, and Smart Attendance Backend swimlanes.

The student selects an active attendance session and requests check-in through the mobile application. The backend validates the attendance session, check-in window, student eligibility, and any previous submission. When check-in is available, the application requests camera and location permissions and obtains the student's current location.

The backend then validates the location against the session geofence. When the student is within the permitted area, the mobile application presents the liveness and face-capture instructions. The student completes the liveness challenge and submits the captured face image, location, and liveness result.

The backend verifies the submitted information and records either a successful result or a failed or pending-review outcome. A successful check-in is marked as Present or Late according to the applicable time threshold. The attendance result and verification log are stored before the final status is displayed to the student.

6.3 Attendance Session Management Activity

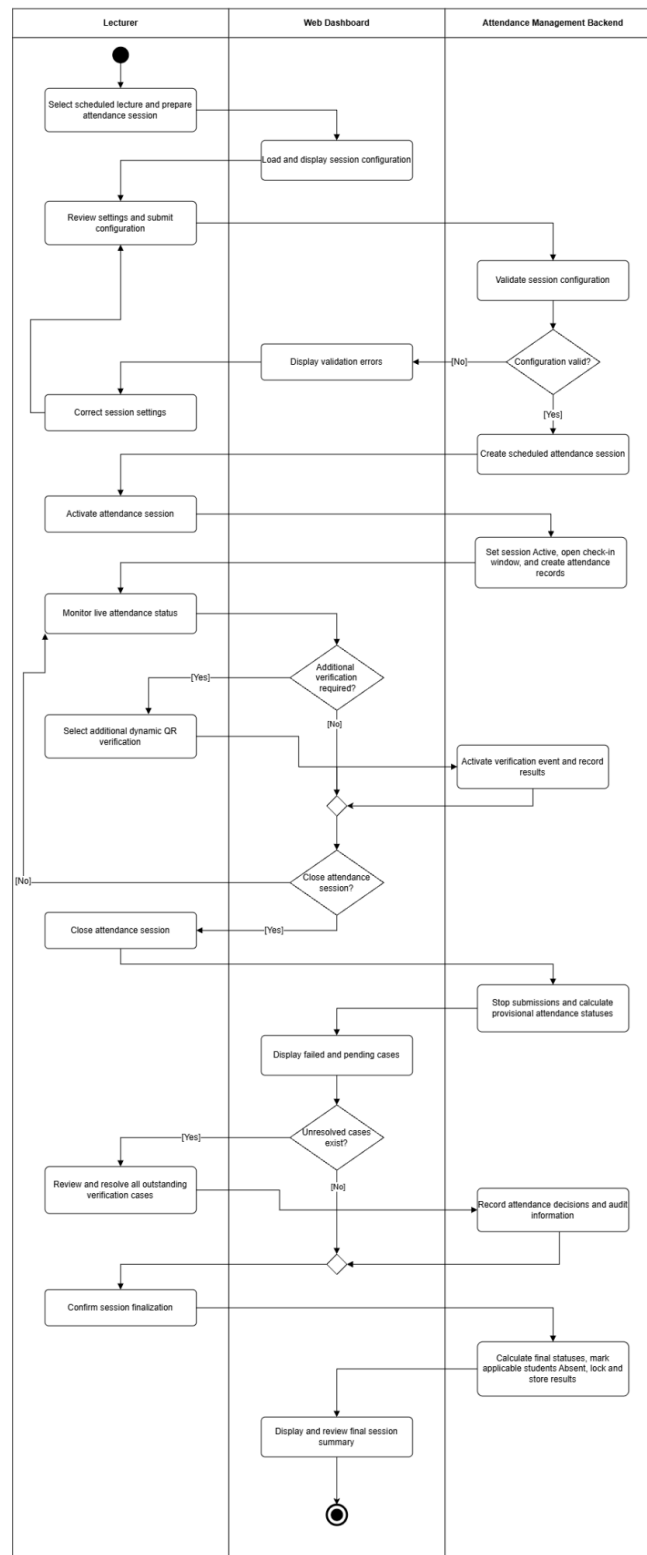


Figure 8. Attendance Session Management Activity Diagram

Figure 8 describes how a lecturer creates, activates, monitors, closes, and finalises an attendance session. The process is divided among the Lecturer, Web Dashboard, and Attendance Management Backend.

The lecturer selects a scheduled lecture and reviews the session configuration. The backend validates the submitted configuration and either creates the scheduled attendance session or returns validation errors for correction.

After activation, the backend opens the check-in window and creates the attendance records required for the session. The lecturer monitors live attendance and may conduct an additional dynamic QR-verification event. When the lecturer closes the session, the backend stops further submissions and calculates provisional attendance statuses.

Failed and pending verification cases are displayed for lecturer review. After unresolved cases have been handled, the lecturer confirms finalisation. The backend then calculates the final attendance statuses, marks applicable students as Absent, locks the records, and stores the results.

6.4 Lecturer Creates Attendance Session Sequence

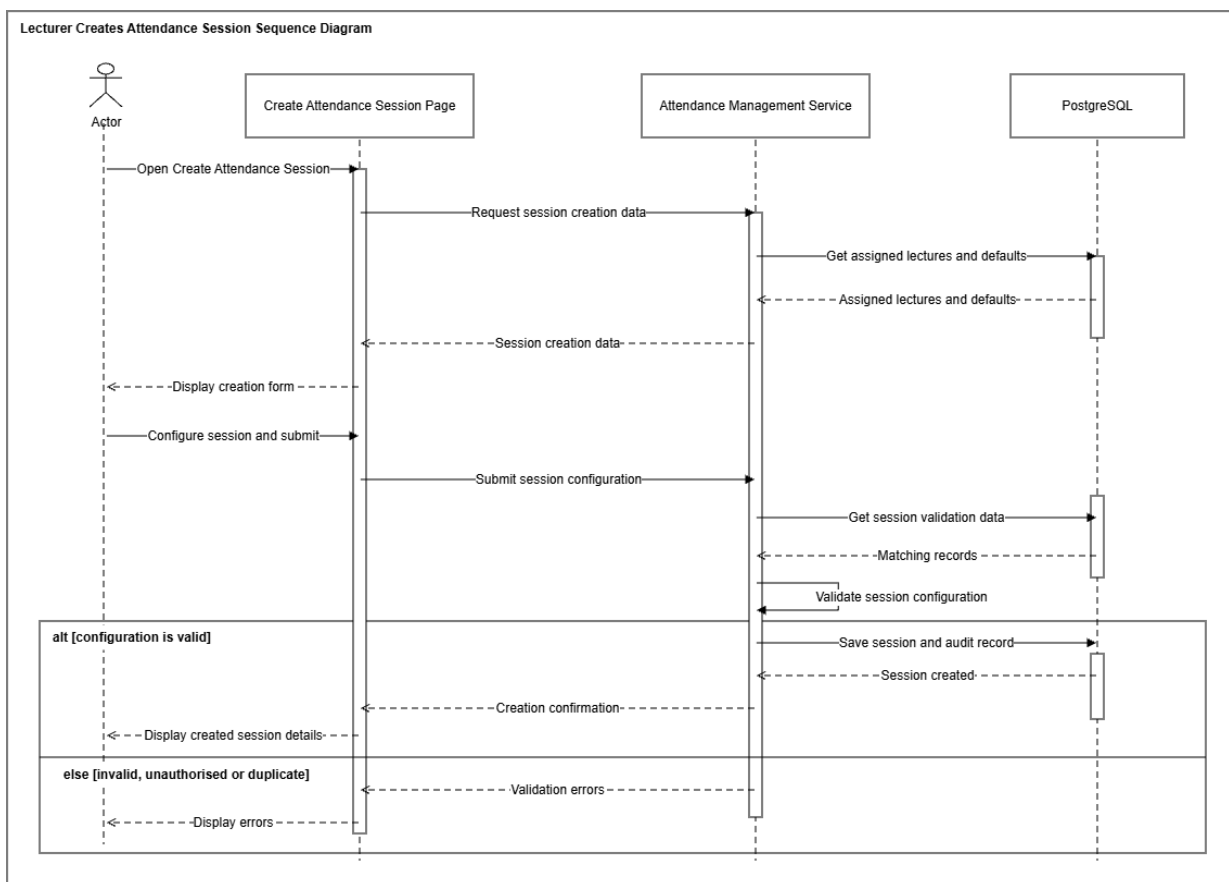


Figure 9. Lecturer Creates Attendance Session Sequence Diagram

Figure 9 shows the interaction among the Actor, Create Attendance Session Page, Attendance Management Service, and PostgreSQL during session creation.

The actor opens the session-creation page, which requests the required creation data from the Attendance Management Service. The service retrieves the assigned lectures and default values from PostgreSQL and returns them to the page.

After the actor configures and submits the session, the service retrieves the relevant validation data and checks the submitted configuration. When the configuration is valid, the service saves the session and its audit record and returns a creation confirmation. When the configuration is invalid, unauthorised, or duplicated, the service returns validation errors for display.

6.5 User Login Sequence

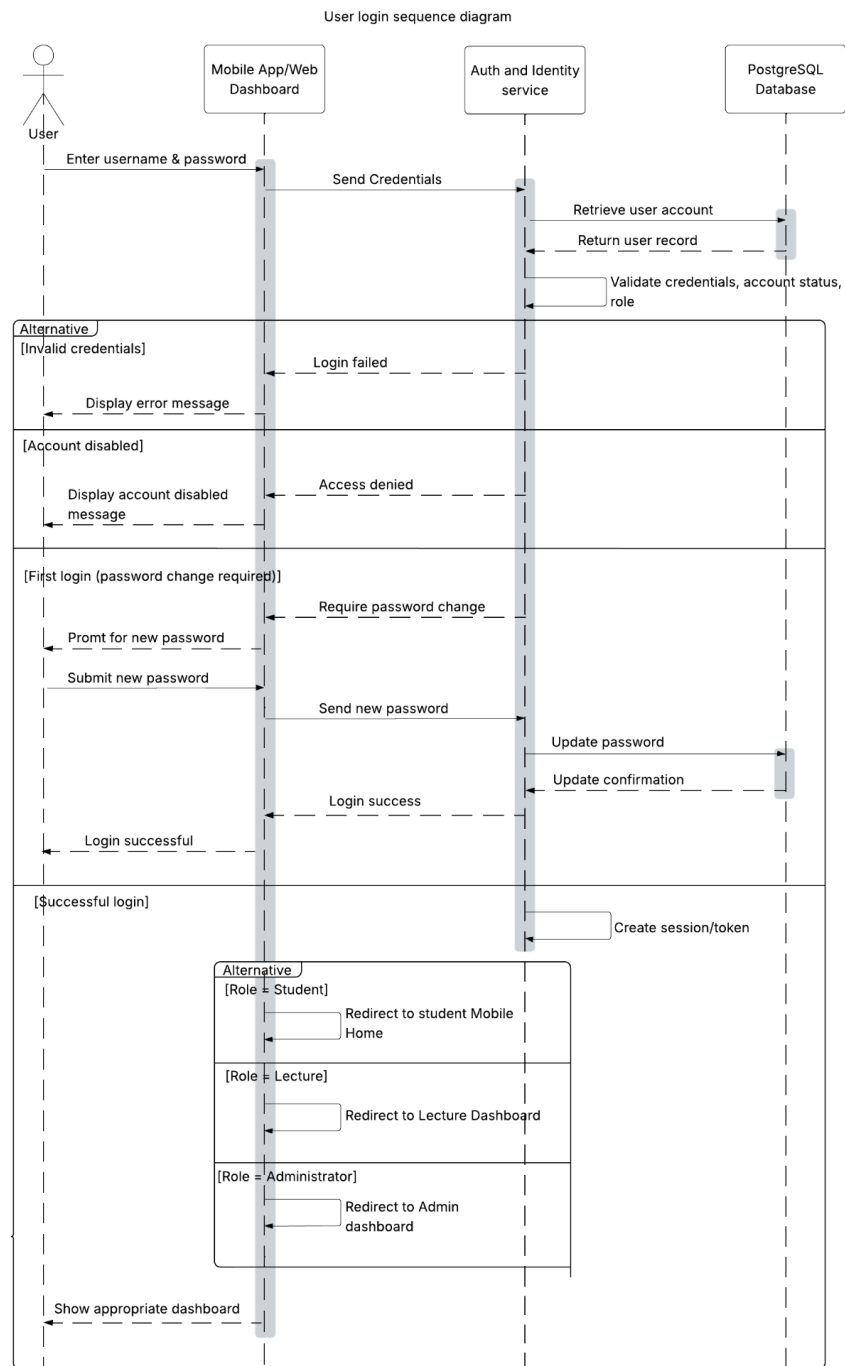


Figure 10. User Login Sequence Diagram

Figure 10 illustrates the login process involving the User, Mobile Application or Web Dashboard, Authentication API, and PostgreSQL Database.

The user submits a username and password through the client application. The Authentication API retrieves the corresponding user account and validates the credentials, account status, and assigned role.

The diagram includes alternative paths for invalid credentials, a disabled account, and a first login that requires a password change. After successful authentication, the API creates the required session or token. The client then redirects the user to the student mobile home, lecturer dashboard, or administrator dashboard according to the assigned role.

6.6 Student Attendance Check-In Sequence

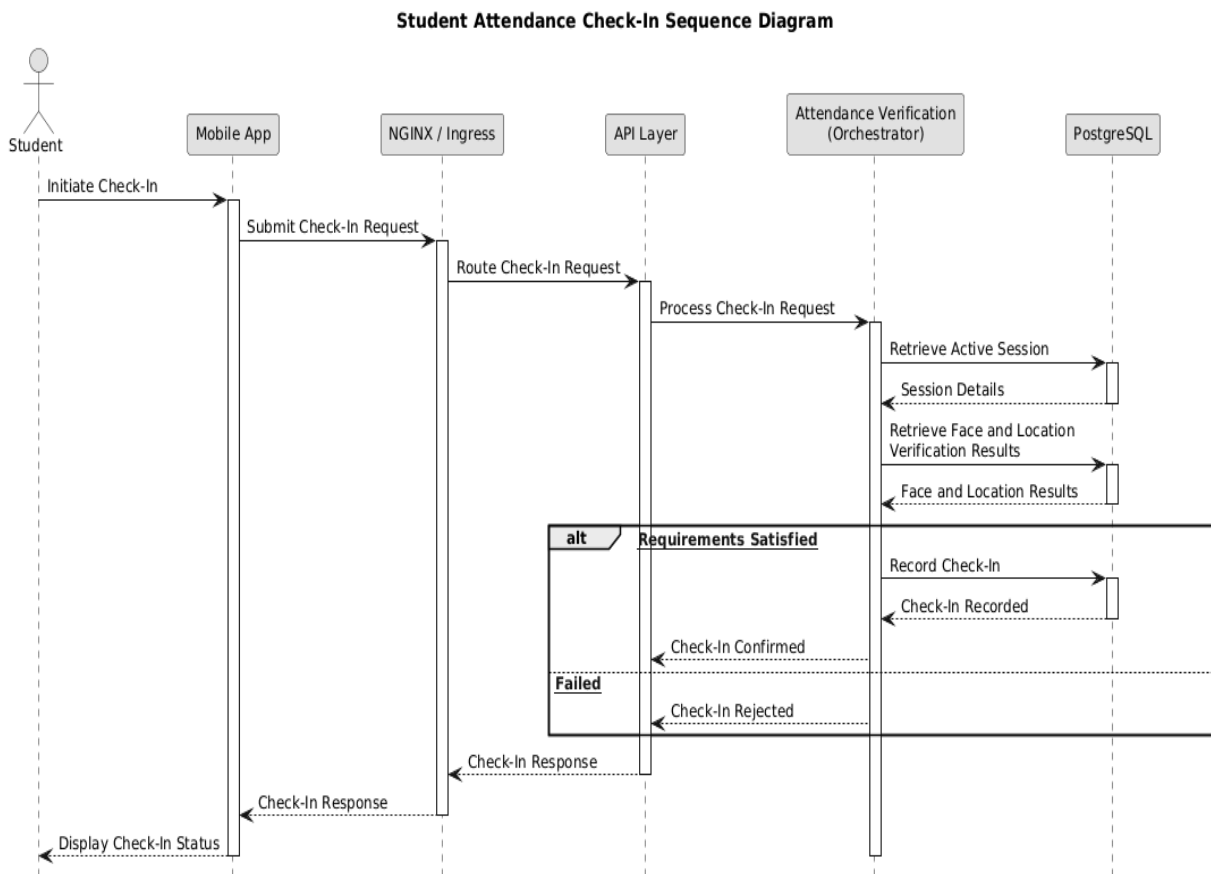


Figure 11. Student Attendance Check-In Sequence Diagram

Figure 11 shows the interactions among the Student, Mobile Application, Backend API, Attendance Service, and Database during attendance check-in.

The student initiates check-in through the mobile application, which submits the request to the Backend API. The request is passed to the Attendance Service, which retrieves the available face, QR, and geofence verification results from the database.

When the verification requirements are satisfied, the Attendance Service evaluates the submission time. A submission within the attendance window is stored as Present, while a submission after the grace period is stored as Late. When one or more verification requirements fail, the attendance request is rejected. The

resulting attendance status is returned through the Backend API and displayed by the mobile application.

The Face Verification, Dynamic QR Validation, and Geofence Validation sequence diagrams provide more detailed views of the validation processes referenced by this sequence.

6.7 Face Verification Sequence

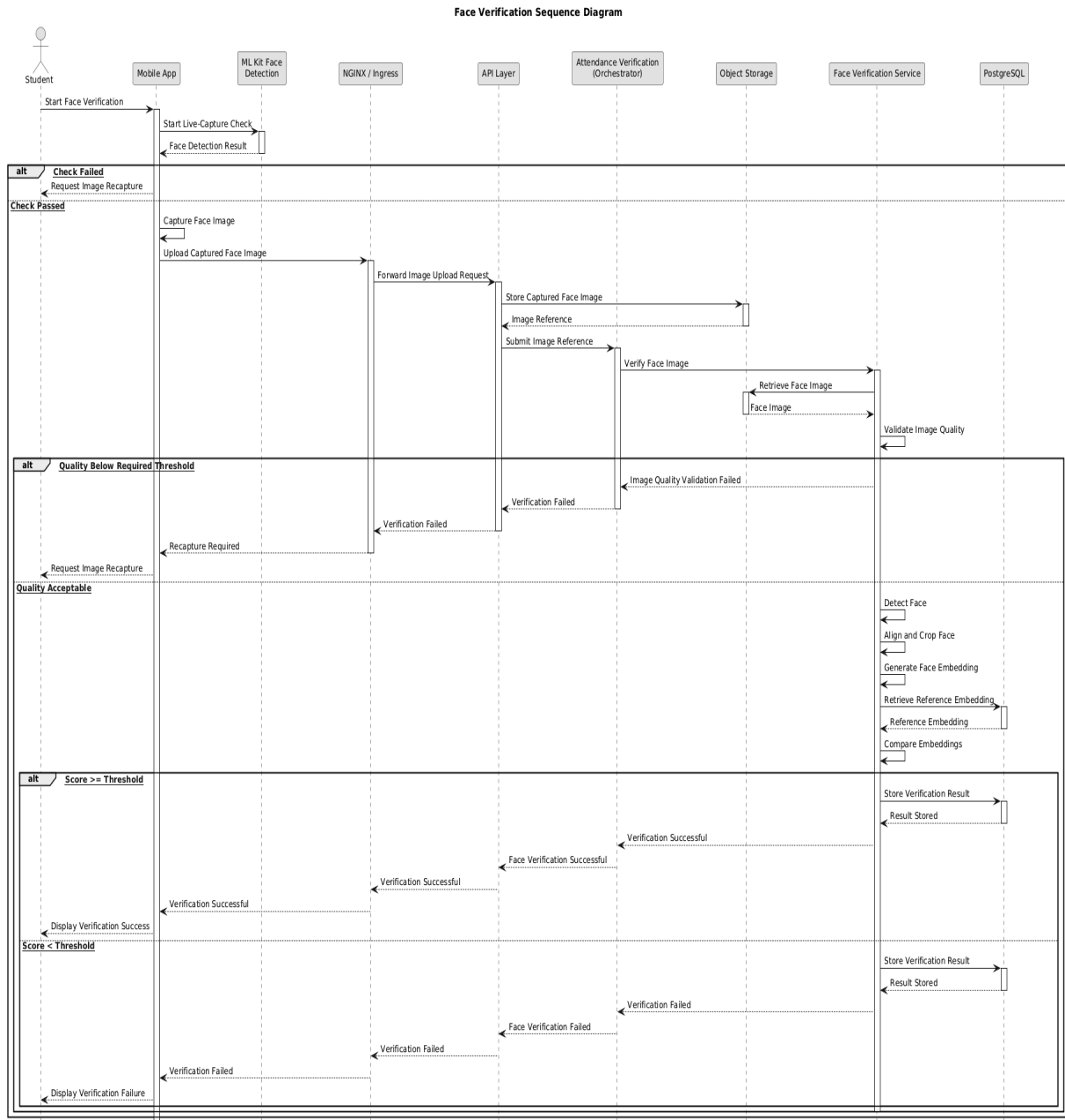


Figure 12. Face Verification Sequence Diagram

Figure 12 describes the face-verification process involving the Student, Mobile Application, Backend

API, Face Service, and Database.

The student starts face verification and completes the liveness challenge presented by the mobile application. The captured face image is submitted to the Backend API, which first validates image quality.

When the image quality is below the required threshold, validation fails and the student is requested to recapture the image. When the quality is acceptable, the Face Service detects and aligns the face, generates a face embedding, retrieves the registered reference embedding, and compares the two embeddings.

When the match score meets or exceeds the configured threshold, a successful verification result is stored and returned to the student. When the score is below the threshold, a failed verification result is stored and the failure is displayed.

6.8 Dynamic QR Validation Sequence

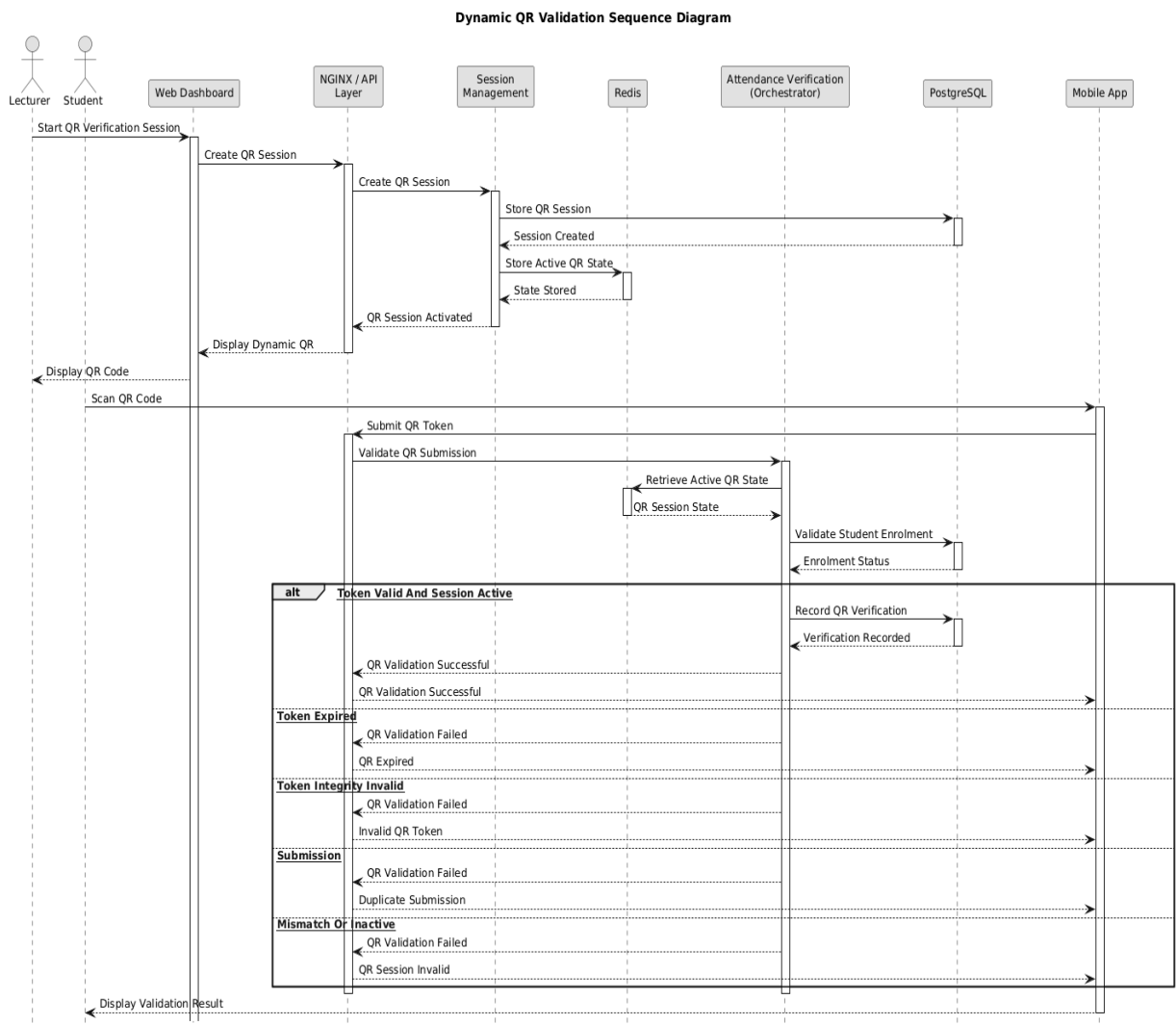


Figure 13. Dynamic QR Validation Sequence Diagram

Figure 13 presents the dynamic QR-verification process involving the Lecturer, Student, Dashboard, Backend API, Database, and Mobile Application.

The lecturer starts a QR session through the dashboard. The Backend API creates the QR session in the database, generates the QR token, and returns it for display. The student scans the displayed QR code using the mobile application and submits the token to the backend.

The backend retrieves the associated QR-session information and validates the submitted token. A token that is valid and not expired produces a successful QR-verification result, which is stored in the database. Expired tokens, duplicate submissions, or tokens that fail integrity validation are rejected, and the corresponding failure result is returned to the mobile application.

6.9 Geofence Validation Sequence

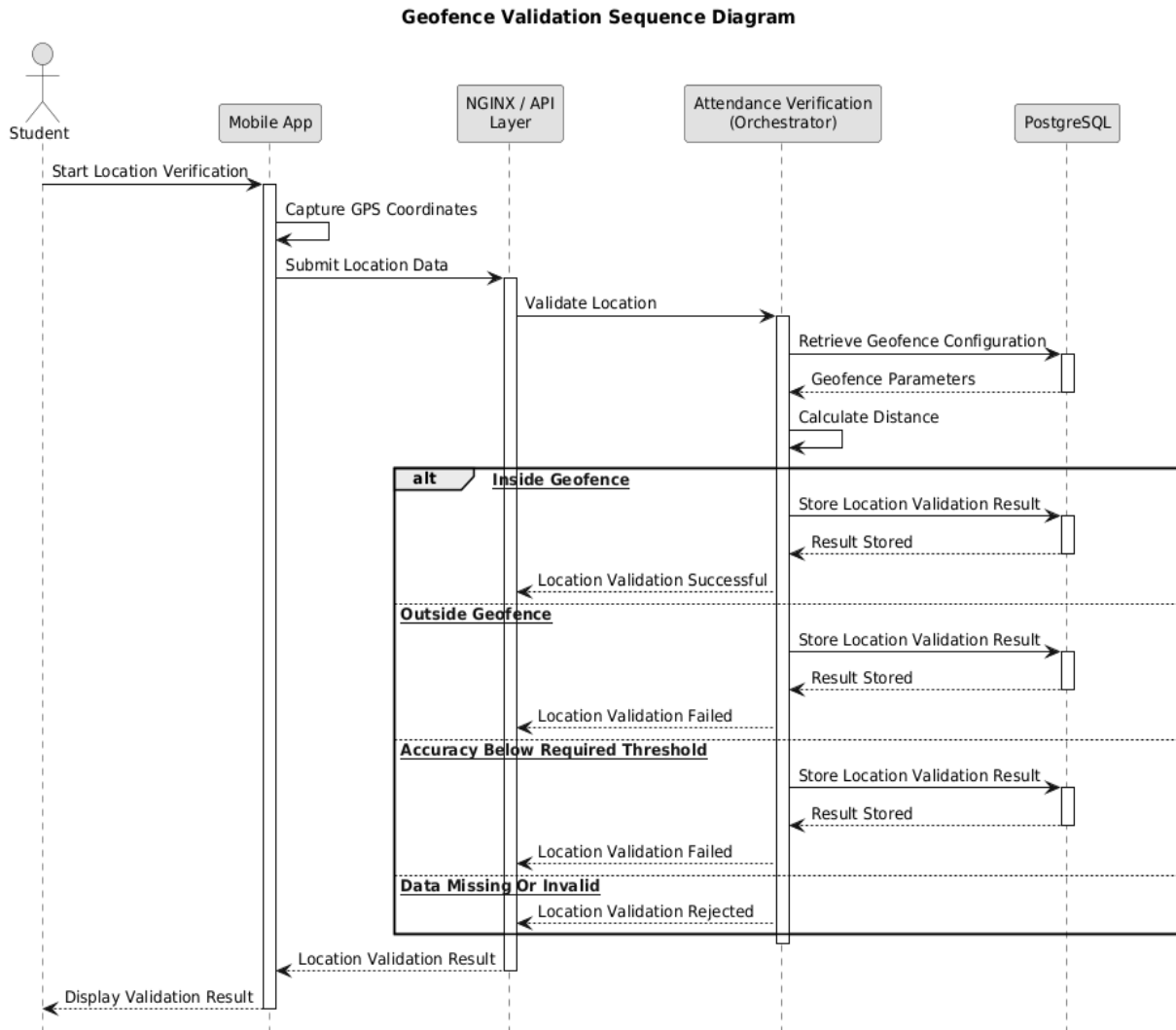


Figure 14. Geofence Validation Sequence Diagram

Figure 14 shows the geofence-validation interaction among the Student, Mobile Application, Backend API, Geofence Service, and Database.

The mobile application captures the student's GPS coordinates and submits the location data to the Backend API. The Geofence Service retrieves the relevant geofence configuration from the database and calculates the distance between the submitted location and the configured geofence.

A location within the geofence produces a successful validation result. A location outside the geofence produces a failed result. When the reported accuracy is below the required threshold, the result requires review. Missing or invalid location information causes the validation request to be rejected. The result is stored where applicable and returned for display to the student.

6.10 Manual Attendance Override Sequence

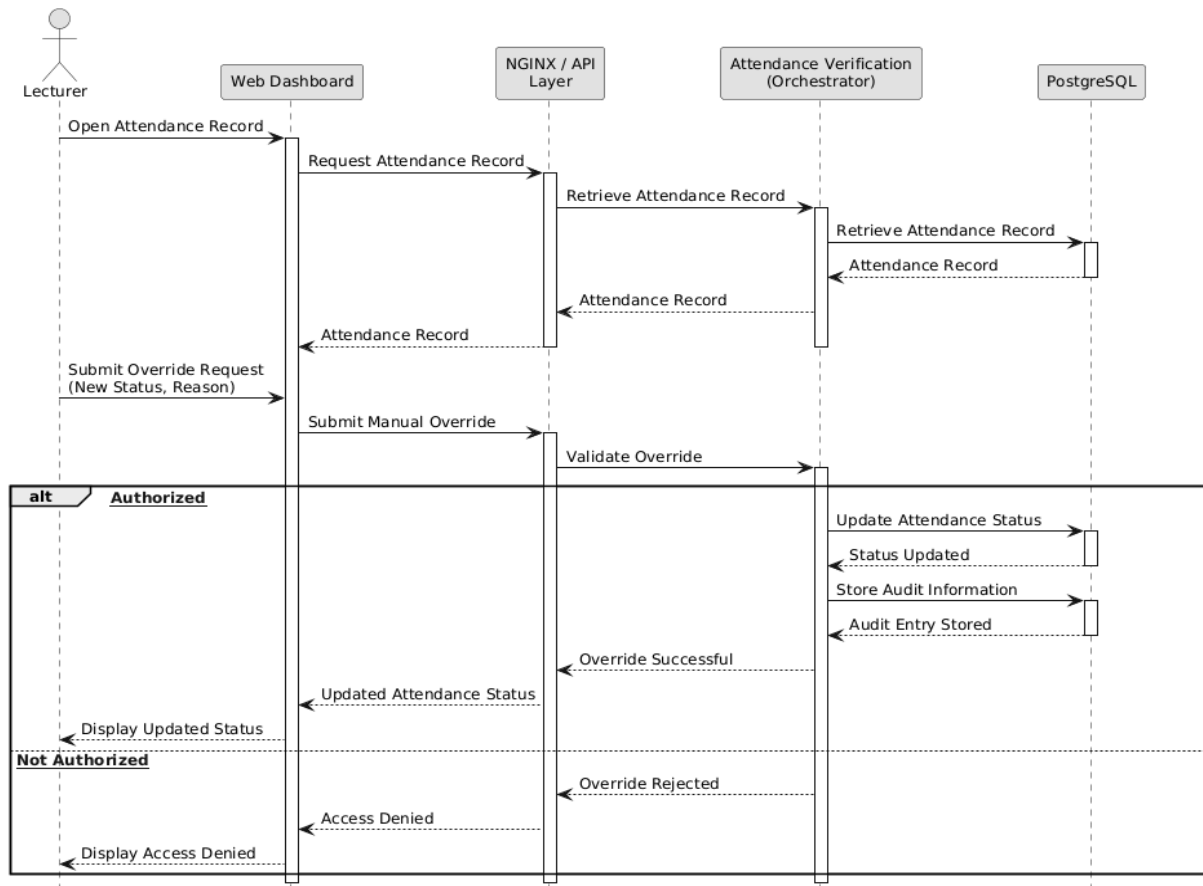


Figure 15. Manual Attendance Override Sequence Diagram

Figure 15 illustrates how a lecturer manually updates an attendance record through the Dashboard, Backend API, Attendance Service, Database, and Audit Log.

The lecturer opens an attendance record, and the system retrieves the relevant information from the database. The lecturer then submits an override request containing the new attendance status and a reason.

The Attendance Service validates whether the lecturer is authorised to perform the update. When authorised, the attendance status is updated in the database and an audit entry is created. The updated status is then displayed to the lecturer. When the lecturer is not authorised, the request is rejected and an access-denied message is returned without changing the attendance record.

6.11 Administrator Manages Academic Data Sequence

6.11.1 Administrator Replaces Student Reference Face

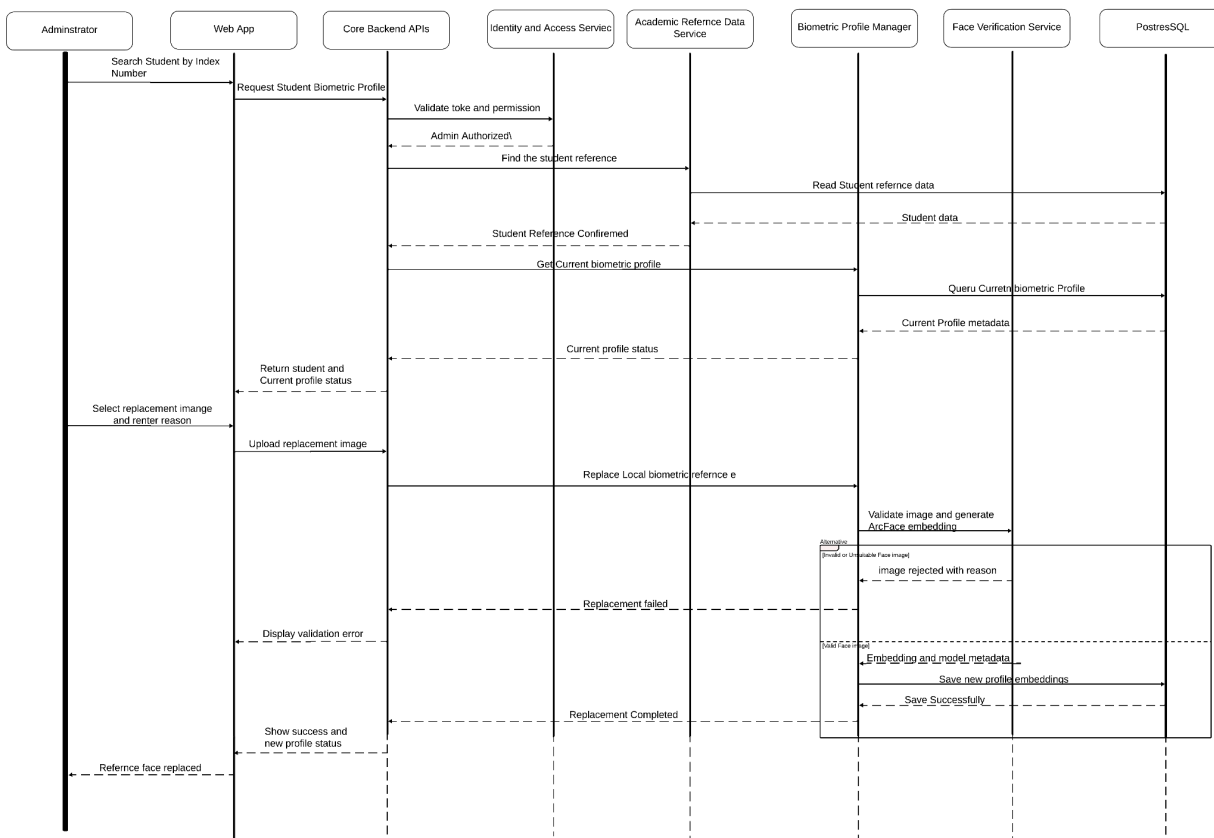


Figure 16.1 Administrator Replaces Student Reference Face

Figure 16.1 describes an administrative operation for replacing a student's reference face. The participating elements include the Administrator, Web Application, Core Backend APIs, Identity and Access Service, Academic Reference Data Service, Biometric Profile Manager, Face Verification Service, and PostgreSQL.

The administrator searches for a student using the index number. The backend verifies the administrator's token and permission, retrieves the student reference data, and obtains the current biometric-profile information.

The administrator selects a replacement image and submits it with the reason for replacement. The Biometric Profile Manager requests the Face Verification Service to validate the image and generate an ArcFace embedding. If the image is rejected, the system returns the relevant validation error. If processing succeeds, the new embedding and model metadata are stored in PostgreSQL, the biometric profile is updated, and the successful replacement is displayed to the administrator.

6.11.2 Administrator Configures Classroom and Geofence

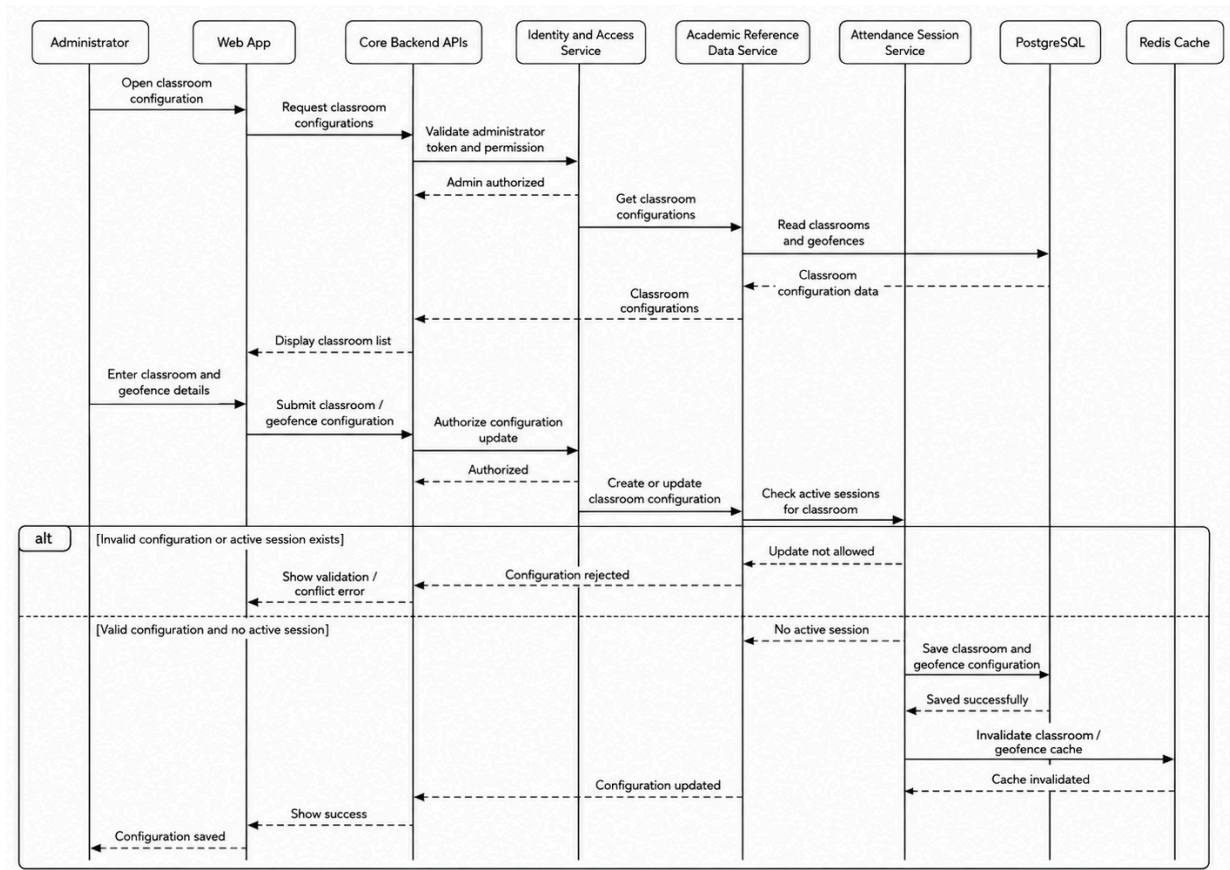


Figure 16.2 Administrator Configures Classroom and Geofence

Figure 16.2 describes the administrative process for configuring a classroom and its geofence settings. The administrator opens the classroom configuration page, and the backend verifies the administrator's token and permission. The system then retrieves the existing classroom and geofence configuration data from PostgreSQL and displays it in the Web Application.

The administrator enters or updates the classroom details, including the geofence configuration, and submits the changes. Before saving the update, the Academic Reference Data Service checks with the Attendance Session Service to determine whether an active attendance session is currently using the classroom. If the configuration is invalid or an active session exists, the update is rejected and the Web Application displays the corresponding validation or conflict message.

If the configuration is valid and there is no active session using the classroom, the updated classroom and geofence configuration is stored in PostgreSQL. The related classroom/geofence cache in Redis is then invalidated so that subsequent requests use the latest configuration. Finally, the updated configuration is returned through the backend and the Web Application confirms the successful change to the administrator.

6.12 Attendance Session State

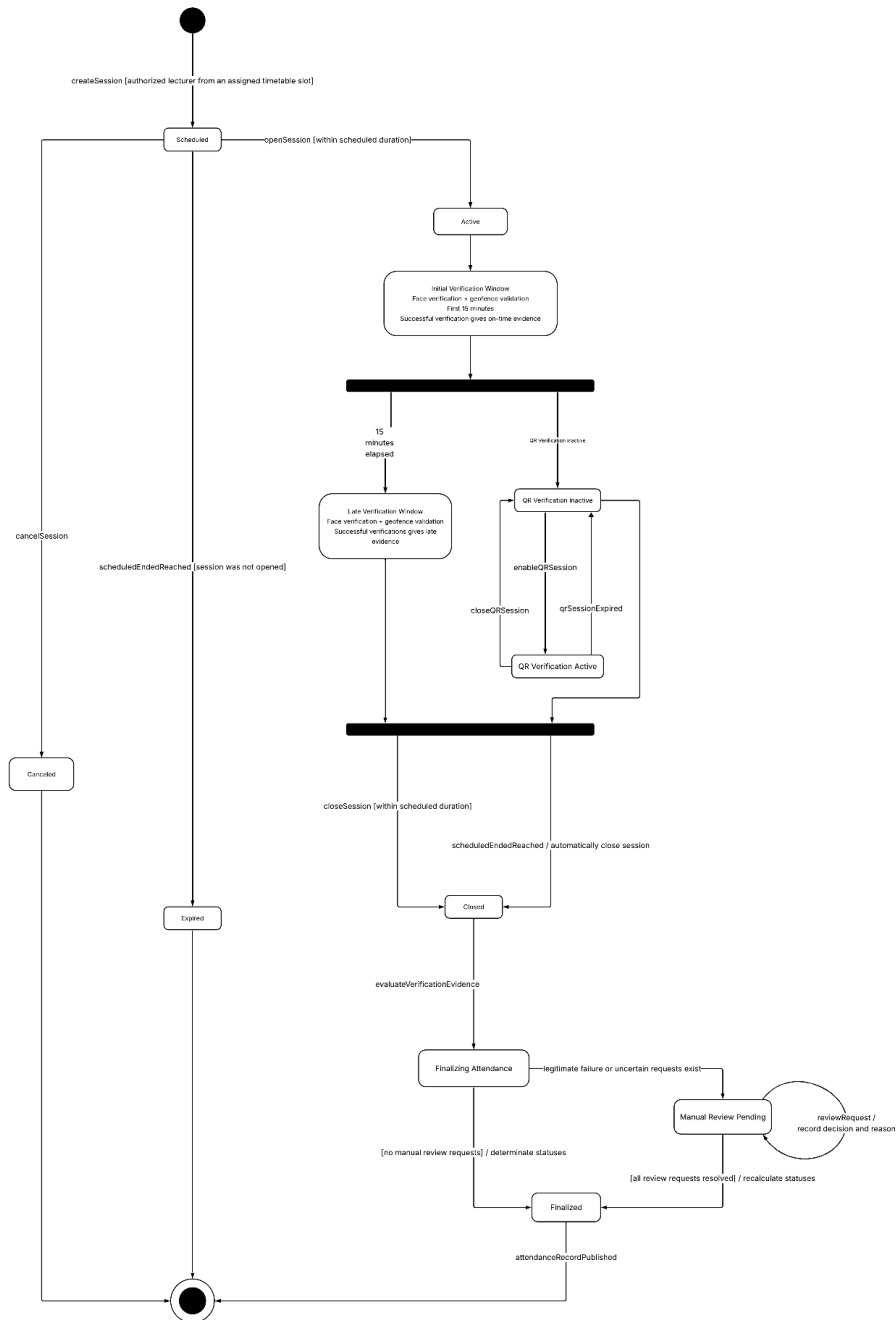


Figure 17. Attendance Session State Diagram

Figure 17 shows the lifecycle of an attendance session. A session begins in the Scheduled state after it is created by an authorised lecturer for an assigned timetable slot.

A scheduled session may be opened within the permitted duration and enter the Active state. Once active, the session enters the Initial Verification Window, during which face verification and geofence validation are performed and successful submissions are recorded as on-time evidence.

After the initial period, the session continues through the Late Verification Window. The lecturer may also enable one or more QR-verification sessions, which move between inactive and active states while the attendance session remains open.

The session enters the Closed state when the lecturer closes it or when the scheduled end time is reached. The system then evaluates the collected evidence during Finalising Attendance. Sessions containing legitimate failures or uncertain cases may move to Manual Review Pending before reaching Finalised. A scheduled session may also be Cancelled or Expired when it is not opened as expected.

6.13 Attendance Record State

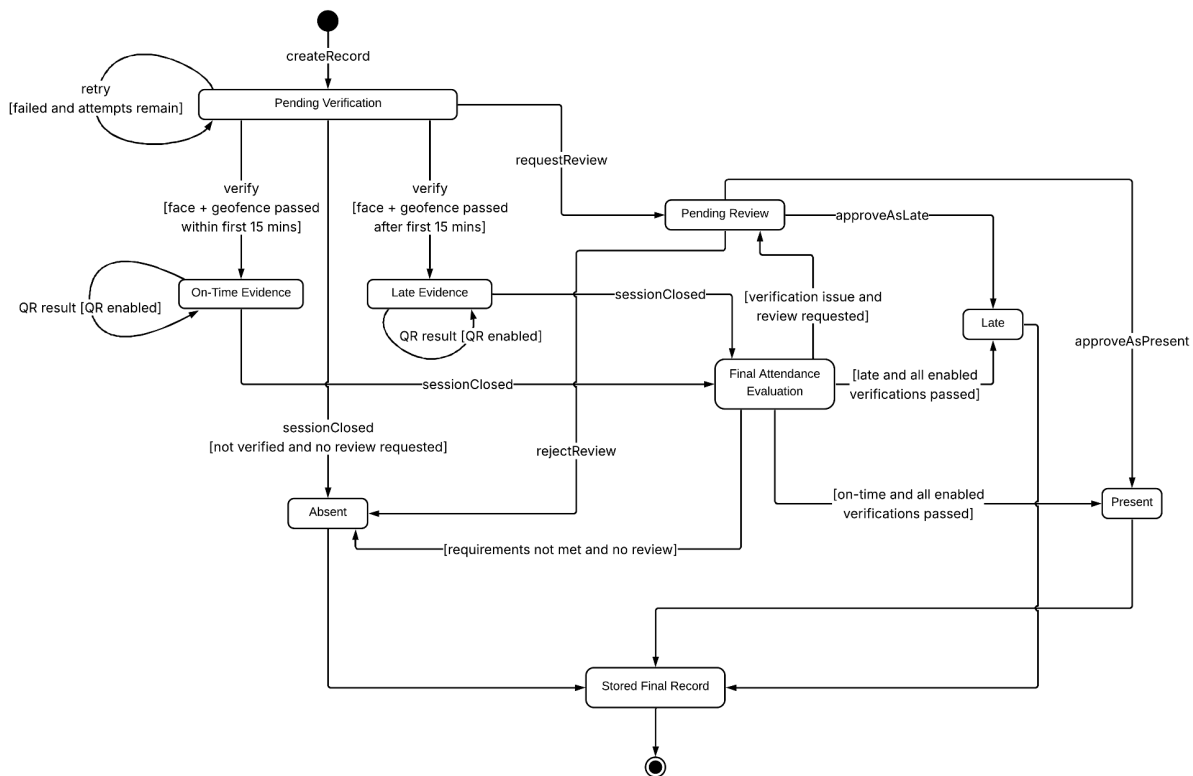


Figure 18. Attendance Record State Diagram

Figure 18 presents the lifecycle of an individual student attendance record. A newly created record begins in Pending Verification. Failed attempts may return to the same state while retry attempts remain.

When face and geofence verification pass within the first 15 minutes, the record receives On-Time Evidence. Verification completed after the first 15 minutes produces Late Evidence. When QR verification is enabled, its result is associated with the available evidence.

A student or system process may request review, moving the record to Pending Review. When the session closes, the available evidence is evaluated during Final Attendance Evaluation. A valid on-time result becomes Present, while valid late evidence becomes Late. A record that does not satisfy the requirements and is not approved through review becomes Absent.

A pending-review case may be approved as Present or Late or rejected as Absent. The selected final status is then written through the Store Final Record state, completing the attendance-record lifecycle.

7. Deployment View

The Deployment View describes the physical and cloud environments in which the system components execute and the communication paths between them. The platform is distributed across student mobile devices, lecturer and administrator devices, cloud-hosted frontend and backend environments, a separate face-verification server, managed data services, temporary object storage, and an external notification service.

The deployment architecture separates the core backend from computationally intensive face verification while keeping the main backend as a modular monolith. This provides clear deployment boundaries without introducing unnecessary complexity.

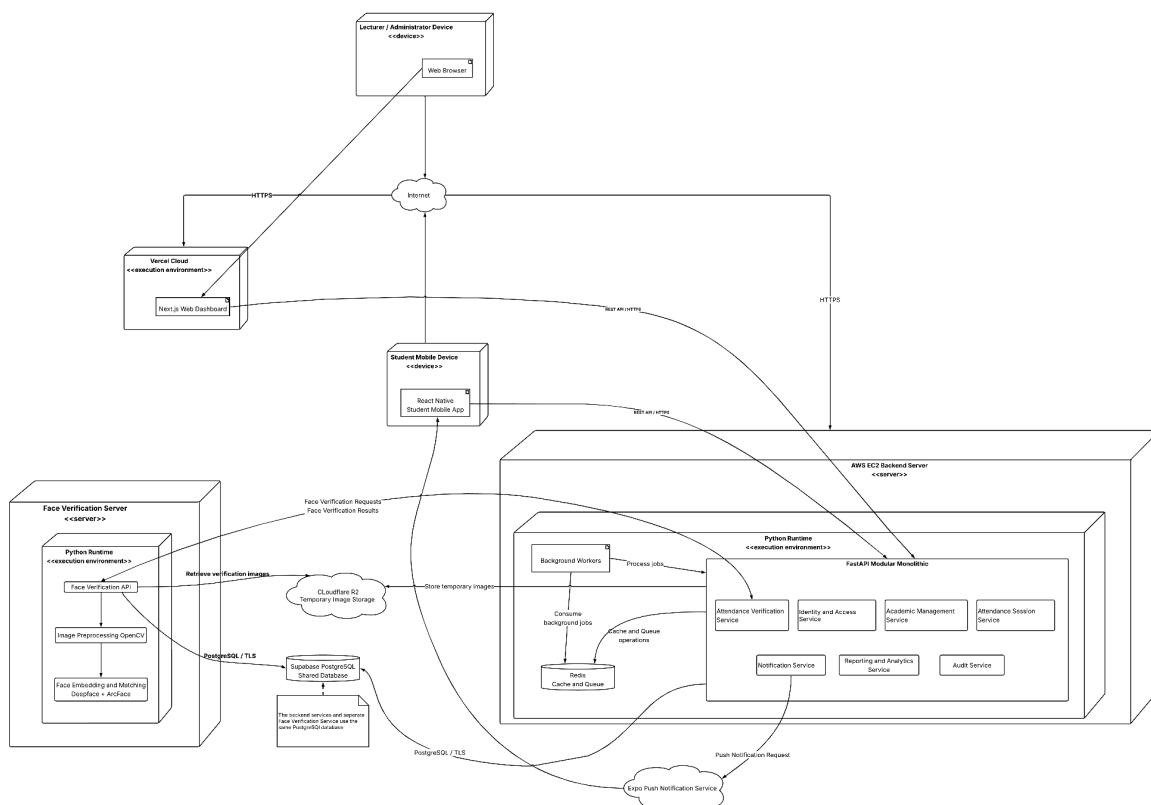


Figure 19. Deployment Diagram

Figure 19 shows the deployment of the Smart Attendance and Classroom Access Platform. Lecturer and administrator users access the Next.js web dashboard through a web browser, while students use the React Native mobile application. Both client applications communicate with the FastAPI backend over the Internet using HTTPS.

The FastAPI backend, Redis, and background workers run on the AWS EC2 backend server. The backend communicates with the separately deployed Face Verification Server, the shared Supabase PostgreSQL

database, Cloudflare R2 temporary object storage, and the Expo Push Notification Service.

The deployment decisions were made according to the different computational, operational, and data-storage requirements of each component.

- **Separate mobile and web applications:** Students require camera, GPS, QR scanning, and mobile notifications, while lecturers and administrators primarily require a browser-based management interface.
- **Vercel for the web frontend:** Provides a deployment environment suited to the Next.js web application and allows the frontend to be deployed independently of the backend.
- **AWS EC2 for the backend:** Provides a controllable environment for running the FastAPI modular monolith, Redis, and background workers without the complexity of independently deploying every backend service.
- **Separate face-verification server:** Face processing requires more computational resources than normal API operations. Separating it prevents intensive verification workloads from affecting the core backend and allows independent scaling if required.
- **Redis:** Provides fast temporary storage, caching, rate limiting, session/QR state, and background-job queuing without using PostgreSQL for short-lived data.
- **Background workers:** Allow operations such as notification delivery and report generation to execute asynchronously without blocking normal API requests.
- **Supabase PostgreSQL:** Provides managed, persistent relational storage for authoritative academic, attendance, verification, and audit data.
- **Cloudflare R2:** Stores temporary face images as objects rather than placing large binary files directly in PostgreSQL.
- **HTTPS:** Protects communication between clients and backend services because requests may contain authentication, location, attendance, and verification information.

Overall, the deployment design balances separation, performance, security, and implementation simplicity. Components with different operational requirements are deployed separately, while closely related backend services remain within the modular monolith. This avoids the operational overhead of a fully distributed microservice architecture while still allowing computationally intensive face verification and supporting infrastructure to be managed independently.

8. Implementation View

8.1 Overview

The Implementation View describes how the logical architecture is realised through software components, implementation layers, and packages. The platform consists of separate client applications, a modular core backend, a separately deployed Face Verification Service, and supporting infrastructure components.

The student mobile application and lecturer/administrator web application provide the user-facing layer. The core backend is implemented using FastAPI and contains the main business services within a single deployable application. The Face Verification Service is implemented and deployed separately behind a defined API boundary because of its computationally intensive processing requirements.

The implementation structure separates user-interface, API, business-logic, data-access, integration, and background-processing responsibilities. Dependencies between these areas are controlled so that

higher-level components access lower-level functionality through defined interfaces rather than directly accessing implementation details.

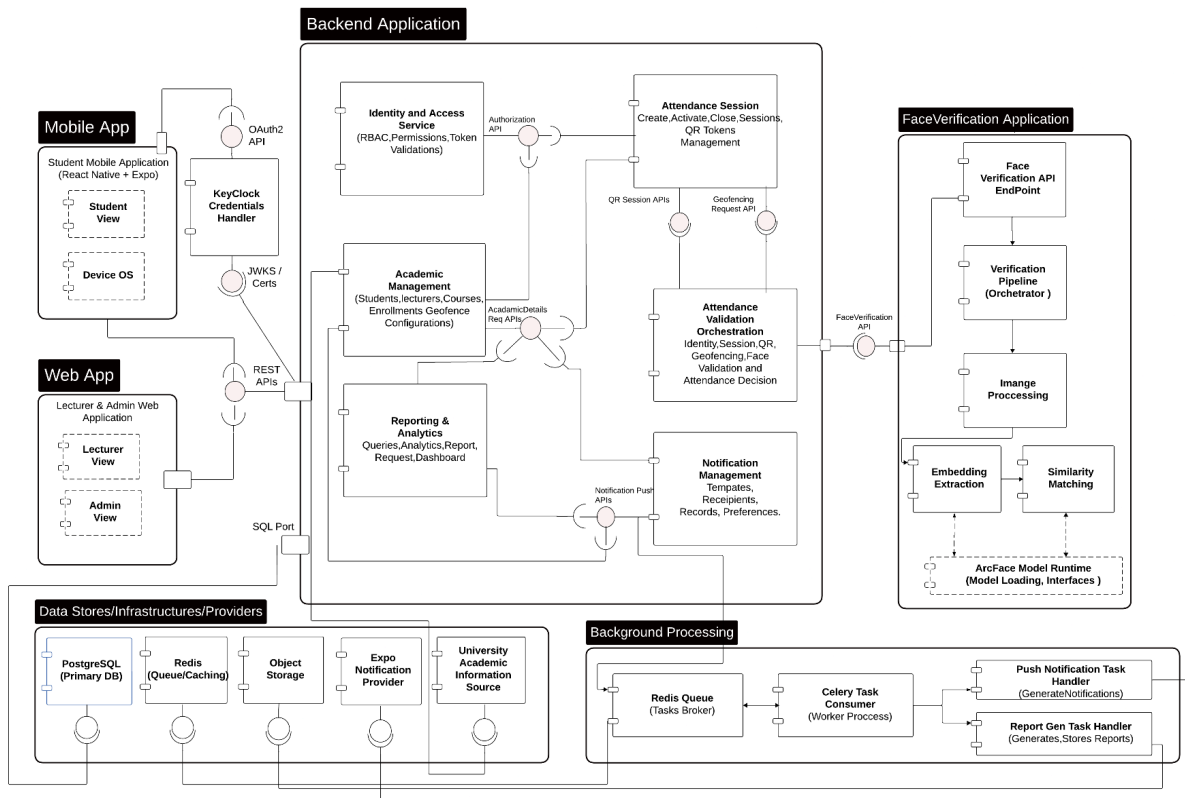


Figure 20. Overall Component Diagram

Figure 20 illustrates the major software components of the platform and the interfaces through which they communicate. It shows the relationship between the mobile and web clients, backend services, face-verification component, data stores, background-processing components, and external services.

8.2 Layers

The implementation is organised into layers according to responsibility. Each layer has a defined purpose and communicates with adjacent layers through established interfaces. This separation reduces coupling between user-interface, business-logic, persistence, and infrastructure concerns.

The main implementation layers are:

Presentation Layer - contains the React Native mobile application and Next.js web dashboard and is responsible for user interaction and presentation.

API Layer - provides the FastAPI endpoints through which client applications access backend functionality. It handles request routing, authentication, authorisation, and input validation.

Business Service Layer - contains the core functional modules responsible for identity and access, academic management, attendance sessions, attendance verification, notifications, reporting and analytics, and auditing.

Data Access Layer - manages access to persistent and temporary application data through PostgreSQL and Redis.

Integration Layer - manages communication with external and separately deployed services, including the Face Verification Service, Cloudflare R2, and notification services.

Background Processing Layer - handles asynchronous tasks using Redis and background workers so that longer-running operations do not unnecessarily block interactive API requests.

The Face Verification Service is maintained outside the core backend layers as a separately deployed service. Its API provides the boundary through which the Business Service Layer communicates with the face-verification functionality.

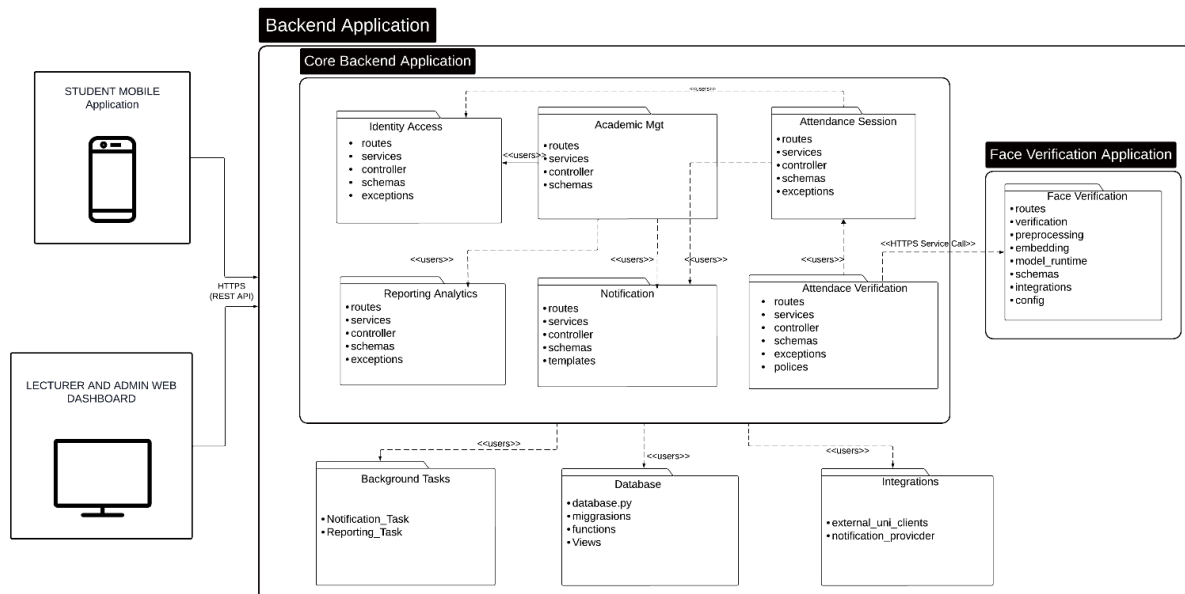


Figure 21. Backend Package Diagram

Figure 21 illustrates the internal organisation of the core backend into packages corresponding to its major functional and technical responsibilities. The packages include identity and access, academic management, attendance sessions, attendance verification, notifications, reporting and analytics, auditing, data access, integrations, and background processing. The dependencies between these packages define how the backend responsibilities interact while maintaining separation between business logic, persistence, and external services.

9. Data View (optional)

9.1 Entity-Relationship Model

The Data View describes the organization, ownership, relationships, and storage of information used by our system.

PostgreSQL is the authoritative persistent data store. The relational model is divided into schemas based on functional responsibility so that closely related tables remain grouped while still participating in a single relational data model.

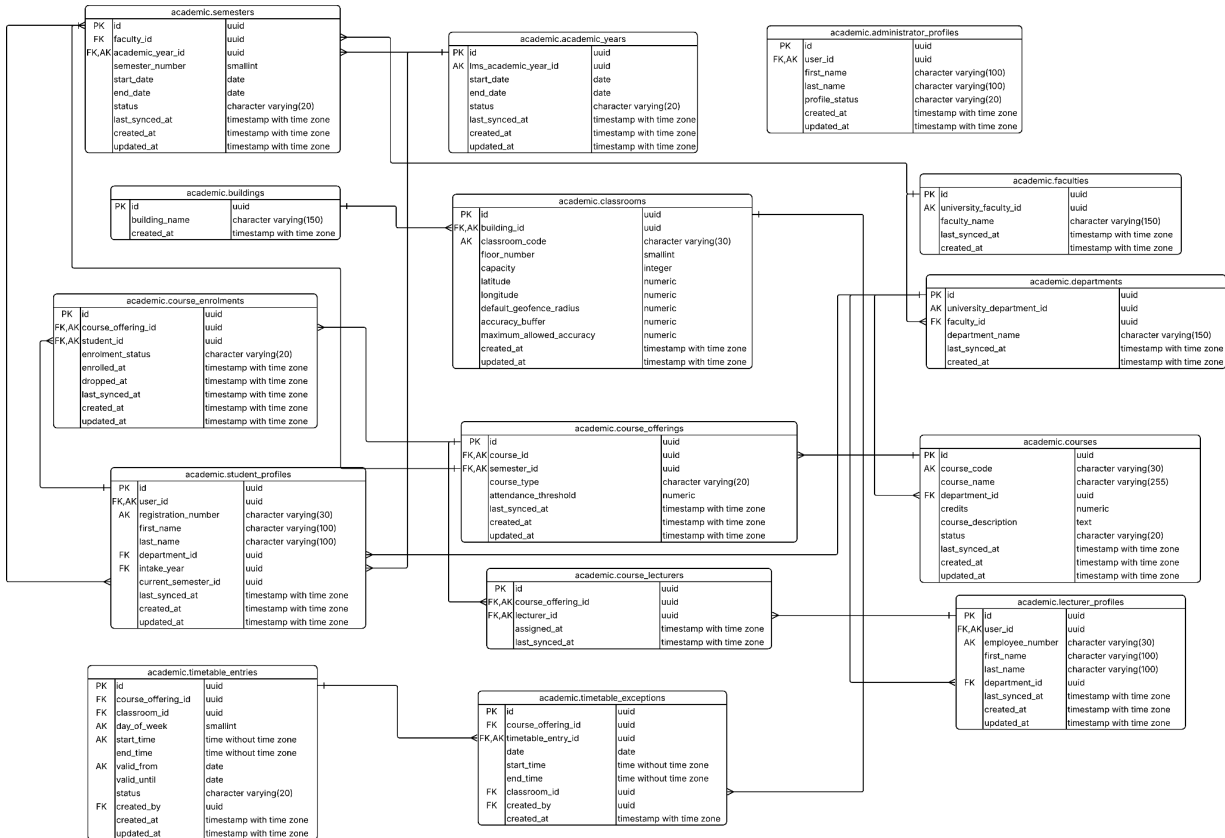


Figure 22. ERD - academic schema

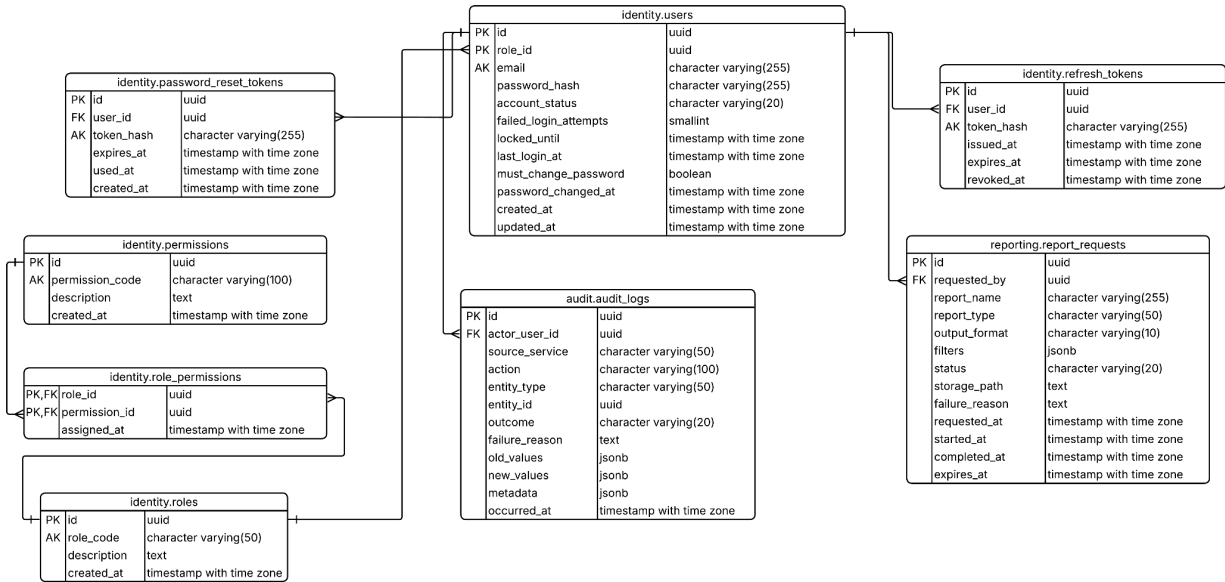


Figure 23. ERD - identity, audit, reporting schemas

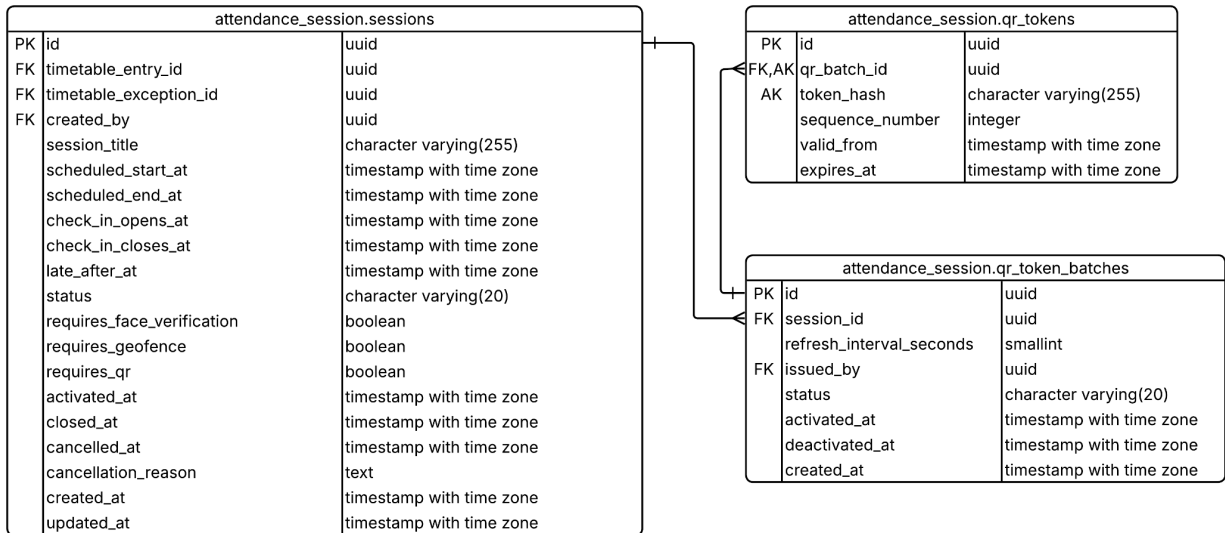


Figure 24. ERD - attendance_session schema

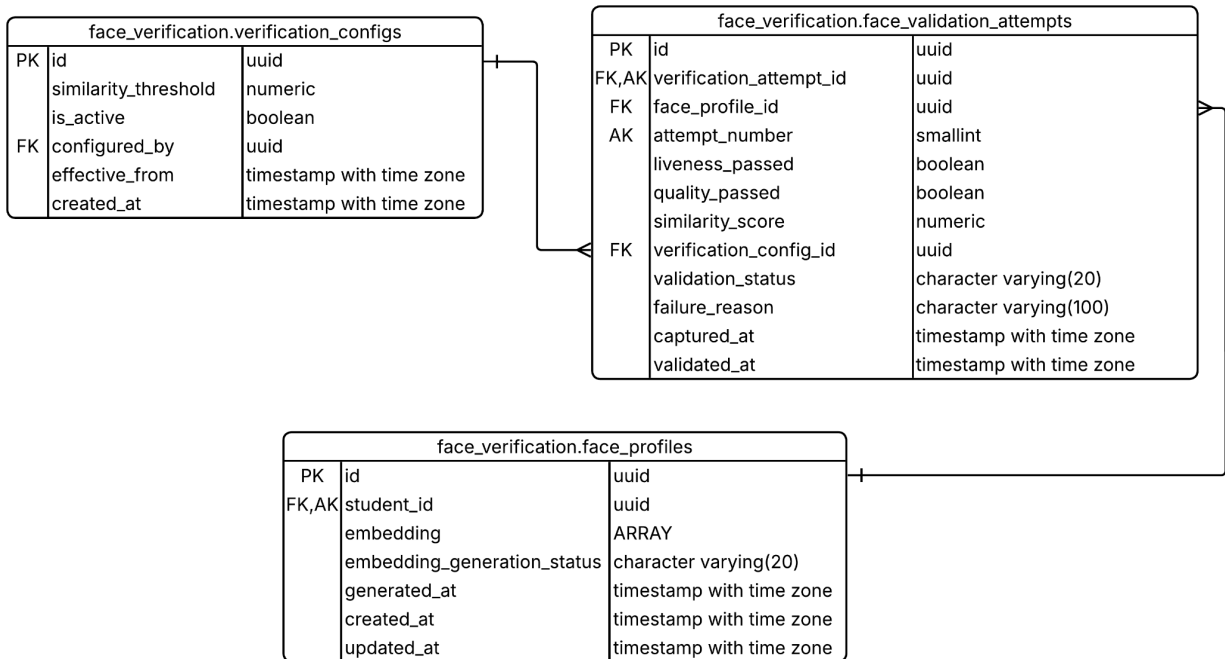


Figure 25. ERD - face_verification schema

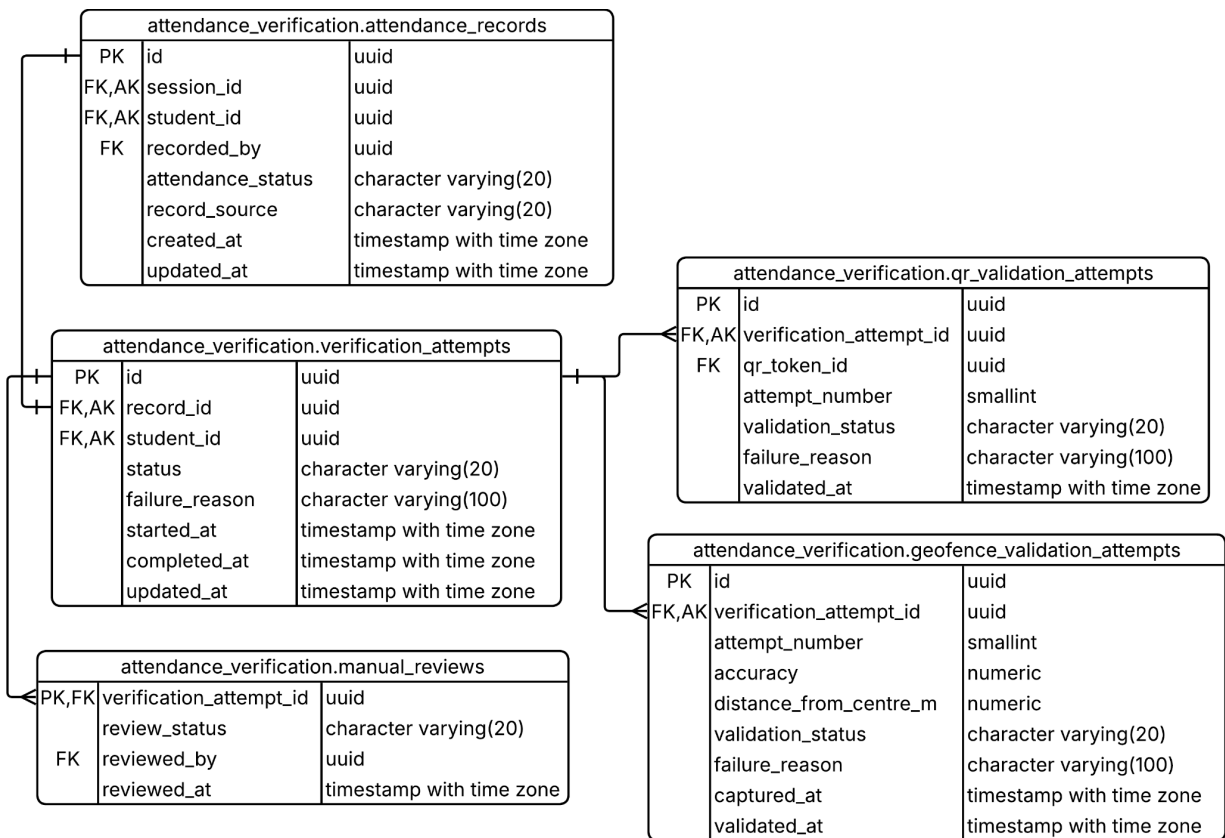


Figure 26. ERD - attendance_verification schema

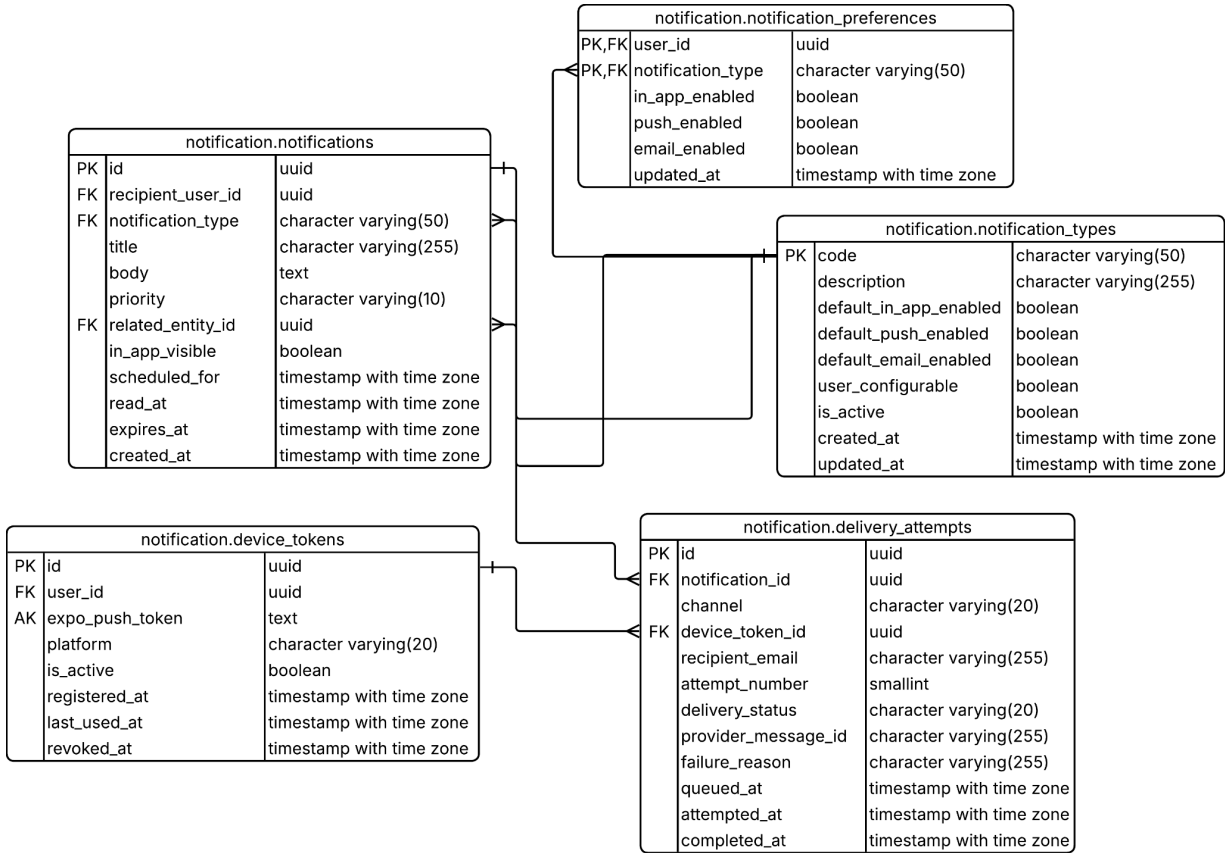


Figure 27. ERD - notification schema

The data model is divided into following schemas

Schema	Responsibility
identity	User accounts, roles, RBAC authentication related other information
academic	Student, lecturer and administrator profiles, organisational structure, courses, course offerings, enrolments, classrooms and timetables
attendance_session	Attendance sessions, eligible students, session geofence configuration and dynamic QR events
attendance_verification	Attendance records, verification attempts, geofence and QR attempts, attendance marking type(by system or manual)
face_verification	Student face profiles, approved reference embeddings, face-validation attempts and verification configuration
notification	User notification records, device tokens, preferences and delivery attempts

reporting	Report requests and generated-report metadata
audit	Immutable records of significant administrative, security and attendance-related actions

A student profile is associated with a system user and may have enrolments in one or more course offerings. Course enrolments establish the many-to-many relationship between students and course offerings. Course offerings are associated with academic periods, lecturers, and timetable information.

Each attendance session is associated with a course offering, classroom, responsible lecturer, timing information, and the verification configuration applicable to that session. The session_students relation links a session with the students eligible to participate in it. This also allows the session roster to be represented independently of later changes to general course-enrolment information.

Dynamic QR verification is represented through QR token batches and individual short-lived tokens associated with an attendance session. QR validation attempts reference the relevant student and session so that successful and failed attempts can be traced without making QR verification a mandatory stage of the initial attendance check-in.

The attendance-verification area stores the evidence produced during attendance processing. Individual verification attempts are preserved separately from the final attendance record. This allows failed or repeated attempts to remain available for audit and review while maintaining one authoritative attendance result for each student and attendance session.

Audit records store significant changes and actions independently from the main operational records. These records support accountability, dispute resolution, and investigation of attendance changes.

Primary keys, foreign keys, unique constraints, check constraints, and indexes are used to maintain relational integrity and improve common access paths.

9.2 Data-Store Responsibilities

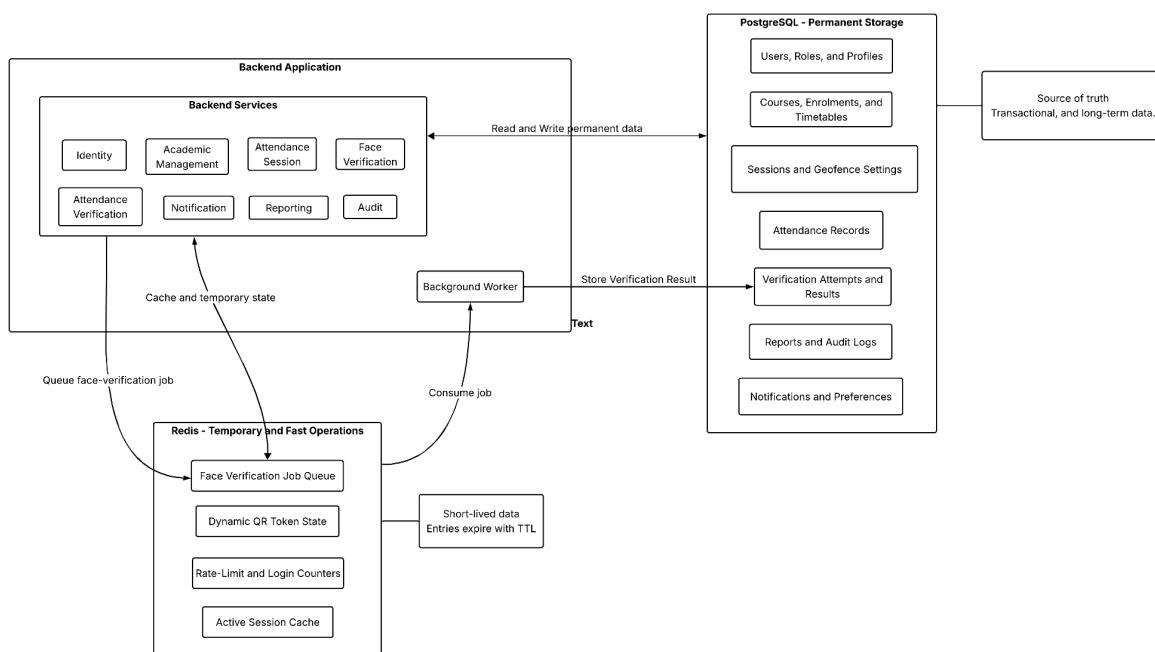


Figure 28. PostgreSQL and Redis Responsibility Diagram

PostgreSQL stores authoritative and persistent application information. This includes user and academic information, course enrolments, timetable information, attendance sessions, final attendance records, verification results, face-profile metadata and embeddings, manual-review records, notification history, report metadata, and audit information. Critical attendance operations use database constraints and transactions to preserve consistency.

Redis stores temporary and recoverable information for performance-sensitive operations. Typical examples include active-session caching, short-lived QR state, rate-limiting counters, repeated-attempt counters, temporary coordination state, and background-job queues. Redis does not act as the system of record. Loss of Redis data must therefore not remove an already finalized attendance record.

Object Storage stores temporary face images required for processing or authorized manual review. Large image objects are kept outside PostgreSQL to prevent unnecessary growth of the relational database. PostgreSQL stores only the necessary metadata and references. Temporary images are deleted after the applicable verification, review, and retention period.

10. Size and Performance

10.1 Overview

The architecture is designed to support concurrent attendance activity during university lecture periods, when a large number of students may attempt to check in within a relatively short time. Performance considerations therefore focus particularly on attendance submission, verification processing, database access, and temporary state management.

The architecture separates computationally intensive face verification from the main backend so that face-processing workloads do not unnecessarily block ordinary API operations.

10.2 Dimensioning Characteristics

The system uses Redis to reduce repeated access to frequently required temporary information such as active session state and rate-limiting information. Background workers are used for operations that do not need to be completed before the user receives the immediate API response.

Database indexing and appropriate query design are used for frequently accessed information such as student identification, session identifiers, enrolments, and attendance records.

Face verification is isolated into a separate service because image processing and embedding comparison require more computational resources than typical API operations. This separation provides an opportunity to allocate additional processing capacity to face verification independently when required.

10.3 Performance Targets

The architecture shall aim to provide:

- Responsive API handling for normal attendance and application operations.
- Low processing overhead for lightweight validations such as session, enrolment, timing, geofence, and QR checks.
- Controlled processing of computationally intensive face verification during peak attendance periods.
- Consistent database response for frequently accessed attendance and session information.
- Asynchronous execution of operations that do not need to delay the immediate user response.
- Independent scaling of face-verification processing when verification demand increases.

Performance targets should be validated through load and integration testing under representative concurrent attendance workloads before production deployment.

11. Quality

The architecture is designed to provide the quality attributes required for dependable operation of the Smart Attendance and Classroom Access Platform. Separation of responsibilities, defined service interfaces, controlled data access, asynchronous processing, and independent deployment of computationally intensive services contribute to the system's security, privacy, reliability, performance, scalability, maintainability, availability, portability, and auditability.

11.1 Security and Privacy

Security and privacy are treated as cross-cutting architectural concerns because the platform processes authentication credentials, attendance records, location information, facial images, and biometric reference data. The architecture applies authentication, role-based authorization, encrypted communication, request validation, rate limiting, controlled access to protected data, and audit logging across the relevant system boundaries.

Temporary facial images are handled separately from persistent academic and attendance data, with controlled access and retention. Location information is used only for attendance-related activities rather than continuous tracking.

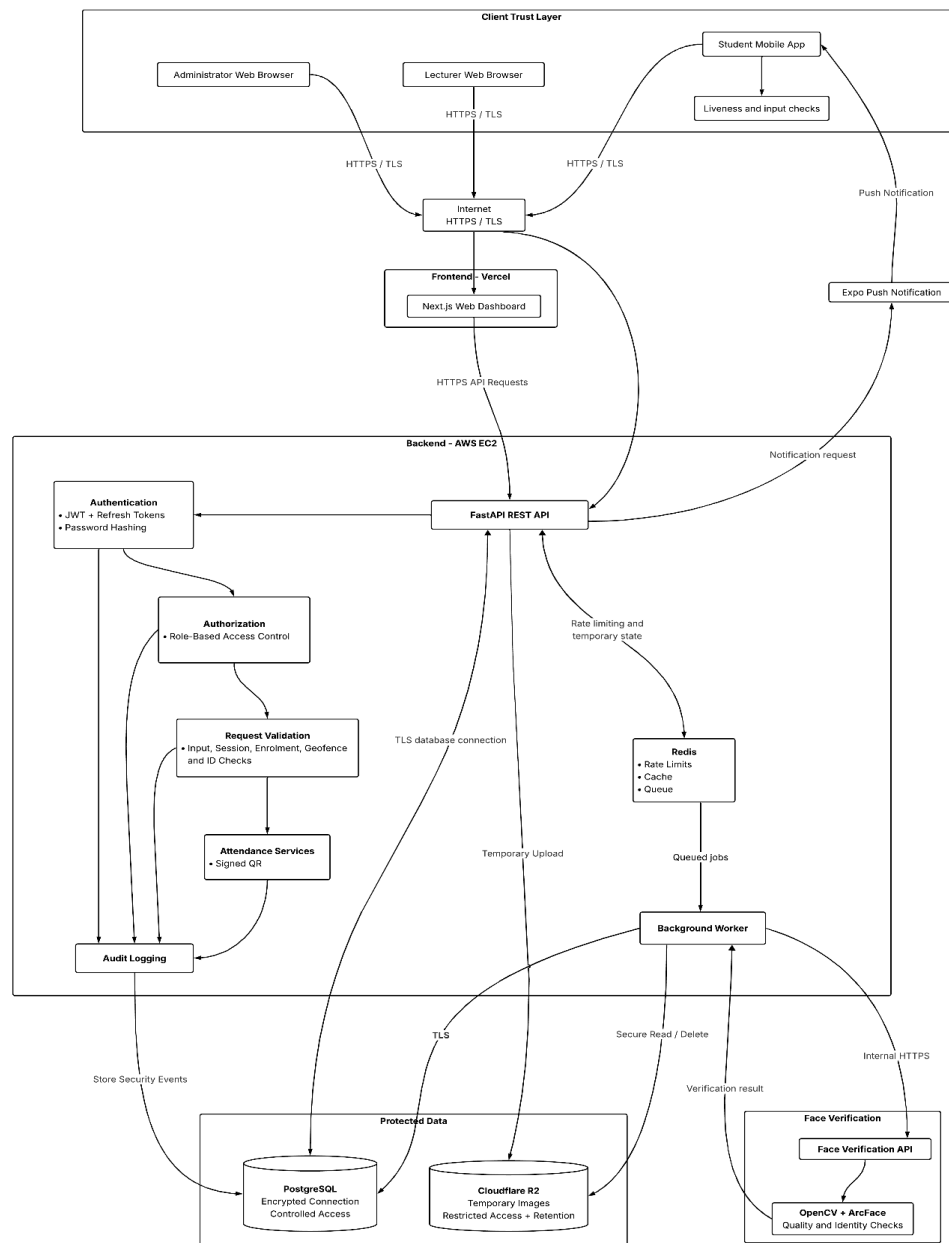


Figure 29. Security Architecture Diagram

Figure 29 illustrates the security boundaries and controls applied across the platform, including authentication, authorization, encrypted communication, request validation, audit logging, protected data storage, and the boundary between the core backend and the separately deployed Face Verification Service.

11.2 Reliability and Availability

The architecture separates authoritative persistent data from temporary processing state. PostgreSQL maintains the system's permanent and transactional records, while Redis handles short-lived state and queued operations. Background processing prevents non-critical operations from blocking interactive attendance requests. The attendance workflow also supports controlled retries and manual review for exceptional verification failures, reducing the likelihood that temporary network, device, location, or verification issues result in incorrect attendance outcomes.

11.3 Performance and Scalability

Performance is supported by separating computationally intensive face verification from the core backend, using Redis for frequently accessed temporary information, and processing suitable operations asynchronously. The independent Face Verification Service can receive additional computing resources without requiring the core backend to be scaled in the same manner.

The modular structure also allows system capacity to be increased as the number of students, attendance sessions, and verification requests grows.

11.4 Maintainability and Extensibility

The core backend is organised into clearly defined functional modules with controlled dependencies and interfaces. This allows individual capabilities to be modified, tested, or extended without requiring changes throughout the system.

The separate Face Verification Service further supports extensibility because its internal verification technology can be improved or replaced while maintaining the interface used by the core attendance-verification functionality.

11.5 Portability

The architecture minimises unnecessary dependence on specific hosting providers by separating deployment configuration from application logic. Environment-specific values, such as database connections, service endpoints, storage configuration, and security credentials, are supplied through deployment configuration rather than being embedded in application code. Combined with standard technologies and service interfaces, this allows individual components to be relocated to compatible hosting environments with limited application changes when operational or infrastructure requirements change.

11.6 Auditability

Important security and attendance-related operations are recorded through audit logging. This provides traceability for actions such as attendance-session changes, verification outcomes, manual attendance updates, and administrative changes. Audit information supports accountability, operational investigation, and resolution of attendance disputes.

12. References

- [1] Object Management Group, “OMG Unified Modeling Language (OMG UML), Version 2.5.1,” Dec. 2017. [Online]. Available: <https://www.omg.org/spec/UML/2.5.1>. [Accessed: Aug. 6, 2026].
- [2] Lucid Software Inc., "Lucidchart," Lucid Software Inc. [Online]. Available: <https://www.lucidchart.com/>. [Accessed: Jul. 31, 2026].
- [3] draw.io, “draw.io Diagramming Application,” [Online]. Available: <https://app.diagrams.net/>. [Accessed: Jul. 31, 2026].
- [4] Vercel, “Next.js Documentation,” Next.js. [Online]. Available: <https://nextjs.org/docs>. [Accessed: Aug. 7, 2026].
- [5] Microsoft, “TypeScript Documentation,” [Online]. Available: <https://www.typescriptlang.org/docs/>. [Accessed: Aug. 7, 2026].
- [6] Meta Platforms, Inc., “React Native Documentation,” [Online]. Available: <https://reactnative.dev/docs/getting-started>. [Accessed: Aug. 7, 2026].
- [7] Expo, “Expo Documentation,” [Online]. Available: <https://docs.expo.dev/>. [Accessed: Aug. 6, 2026].
- [8] S. Ramírez, “FastAPI,” FastAPI Documentation. [Online]. Available: <https://fastapi.tiangolo.com/>. [Accessed: Aug. 7, 2026].
- [9] PostgreSQL Global Development Group, “PostgreSQL Documentation,” [Online]. Available: <https://www.postgresql.org/docs/current/>. [Accessed: Aug. 5, 2026].
- [10] Redis, “Redis Documentation,” [Online]. Available: <https://redis.io/docs/latest/>. [Accessed: Aug. 5, 2026].
- [11] OpenCV, “OpenCV Documentation,” [Online]. Available: <https://docs.opencv.org/>. [Accessed: Aug. 9, 2026].
- [12] S. I. Serengil, “DeepFace: A Lightweight Face Recognition and Facial Attribute Analysis Library for Python,” GitHub. [Online]. Available: <https://github.com/serengil/deepface>. [Accessed: Aug. 9, 2026].
- [13] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 4690–4699.
- [14] Google, “Face Detection | ML Kit,” Google for Developers. [Online]. Available: <https://developers.google.com/ml-kit/vision/face-detection>. [Accessed: Aug. 9, 2026].
- [15] Amazon Web Services, “What is Amazon EC2?,” *Amazon Elastic Compute Cloud User Guide*. [Online]. Available: <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/concepts.html>. [Accessed: Aug. 9, 2026].
- [16] Supabase, “Database,” Supabase Documentation. [Online]. Available: <https://supabase.com/docs/guides/database/overview>. [Accessed: Aug. 5, 2026].

- [17] Cloudflare, “Cloudflare R2,” Cloudflare Developers. [Online]. Available: <https://developers.cloudflare.com/r2/>. [Accessed: Aug. 5, 2026].
- [18] Vercel, “Vercel Documentation,” [Online]. Available: <https://vercel.com/docs>. [Accessed: Aug. 9, 2026].
- [19] Expo, “Expo Push Notifications: Overview,” [Online]. Available: <https://docs.expo.dev/push-notifications/overview/>. [Accessed: Aug. 6, 2026].
- [20] Expo, “EAS Build,” Expo Documentation. [Online]. Available: <https://docs.expo.dev/build/introduction/>. [Accessed: Aug. 6, 2026].
- [21] Celery Project, “Celery: Distributed Task Queue,” Celery Documentation. [Online]. Available: <https://docs.celeryq.dev/>. [Accessed: Aug. 9, 2026].
- [22] E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3,” RFC 8446, Internet Engineering Task Force, Aug. 2018. [Online]. Available: <https://www.rfc-editor.org/info/rfc8446/>. [Accessed: Aug. 9, 2026].